

PharmaChain-DB

CORE DATABASE DESIGN FOR PHARMACEUTICAL MANUFACTURING

Developed by:

Urvish Babariya

 **GitHub:** github.com/Urvish2007/PharmaChain-DB

 **LinkedIn:** linkedin.com/in/Urvish-Babariya

 **Email:** urvishbabariya501@gmail.com

Contents

1	Executive Summary	3
1.1	Comprehensive Entity Dictionary & Use Cases	3
2	Entity-Relationship Diagram	6
3	Relational Diagram (Created using DIA)	7
4	Relational Diagram (Generated via PostgreSQL)	8
5	Normalization and Functional Dependencies	9
5.1	Master Registries	9
5.1.1	Table: Material_Master	9
5.1.2	Table: Account_Master	9
5.1.3	Table: Employee_Master	10
5.1.4	Table: Equipment_Master	10
5.1.5	Table: Product_Master	10
5.2	Logistics and Financials	11
5.2.1	Table: Transactions	11
5.2.2	Table: Supplier_Contract	11
5.2.3	Table: Warehouse	11
5.2.4	Table: RM_Transaction	12
5.3	Manufacturing Floor	12
5.3.1	Table: Formula_Master	12
5.3.2	Table: Batch	12
5.3.3	Table: Material_Dispensing	13
5.3.4	Table: Production_Log	13
5.4	Quality Assurance and Compliance	13
5.4.1	Table: Material_Quality_Check	13
5.4.2	Table: Product_Quality_Check	14
5.4.3	Table: QC_Audit_Log	14
5.4.4	Table: Maintenance_Log	14
5.5	Commercial Operations	15
5.5.1	Table: FG_Transaction	15
5.5.2	Table: Product_Recall	15
6	Data Definition Language (DDL) Scripts	16
7	Data Manipulation Language (DML) Scripts	21
8	Relevant SQL Queries & Enterprise Analytics	34
9	Active Database Automation: Triggers	44
9.1	1. The Warehouse Automator (trg_deduct_stock_on_dispense)	44
9.2	2. The FDA Compliance Blocker (trg_prevent_bad_sales)	46
9.3	3. The Timeline Enforcer (trg_strict_batch_dates)	49
9.4	4. The Silent Auditor (trg_audit_qc_changes)	51

10 Data Abstraction: Enterprise Views	53
10.1 1. Procurement Dashboard: Low Stock Alerts (<code>v_inventory_shortage</code>)	53
10.2 2. FDA End-to-End Batch Traceability (<code>v_fda_batch_traceability</code>) .	55
10.3 3. Expiry Risk & Inventory Liability Dashboard (<code>v_inventory_expiry_risk</code>)	57
11 Complex Transactions: Stored Procedures	60
11.1 1. The Emergency Quarantine Protocol (<code>execute_product_recall</code>) . .	60
12 BCNF Verification Table	63
13 Future Scope and Next Steps	67

1 Executive Summary

PostgreSQL and PL/pgSQL, this project demonstrates strong **backend database design**, **secure data handling**, and **automation** for pharmaceutical manufacturing systems.

The main goal of **PharmaChain-DB** is to move important **business logic** directly into the **database** instead of depending heavily on **application code**. The system uses smart **database triggers**, well-structured **normalized tables**, and **stored procedures** to automatically enforce **FDA compliance rules**, manage **real-time inventory**, and handle **emergency product recalls** safely and efficiently.

1.1 Comprehensive Entity Dictionary & Use Cases

To accurately model a highly regulated pharmaceutical factory, the database is divided into specialized tables across five distinct operational zones. Here is the complete breakdown of every entity, its role, and its business value:

1. Master Registries (The Ground Truth)

These tables act as the foundational dictionaries of the system. Nothing can happen in the factory unless it is first registered here.

- **Material_Master**: The catalog of raw chemical ingredients (APIs and Excipients).
Use Case: Tracks critical safety data like hazardous status and storage conditions before chemicals ever enter the building.
- **Account_Master**: The directory of all external business partners (Suppliers, Distributors, Hospitals).
Use Case: Links financial invoices to real-world companies so the business knows exactly who they are buying from and selling to.
- **Employee_Master**: The directory of factory staff and their roles.
Use Case: Provides accountability. Every lab test or machine operation is permanently tagged to a specific employee's ID.
- **Equipment_Master**: The registry of all physical machines on the factory floor.
Use Case: Tracks calibration dates and maintenance statuses to prevent production from running on broken equipment.
- **Product_Master**: The official catalog of finished medicines the company produces.
Use Case: Ensures the factory only manufactures approved, legally registered drugs.

2. Financials & Logistics

These tables handle the money, legal agreements, and corporate accounting.

- **Transactions**: The unified financial ledger for both buying and selling.
Use Case: Centralizes all corporate accounting into one place to easily track cash flow, invoice statuses, and revenue without data duplication.

- **Supplier_Contract**: The legal agreements locked in with raw material vendors.
Use Case: Prevents the purchasing team from buying chemicals at the wrong price or using an expired legal contract.

3. Inventory Pipeline

These tables manage the physical storage and safety screening of incoming raw materials.

- **Warehouse**: The real-time physical storage tracker utilizing serialized unique key pairing.
Use Case: Gives managers an exact count of what is on the shelves, enabling automated alerts when stock drops below safe reorder levels.
- **RM_Transaction (Raw Material)**: The link between a purchased raw material and its financial invoice.
Use Case: Allows auditors to calculate exactly how much money is tied up in raw chemicals sitting in the warehouse.
- **Material_Quality_Check**: Laboratory test results for newly purchased raw chemicals.
Use Case: Stops contaminated, expired, or low-purity chemicals from ever reaching the factory mixing machines.

4. Production Floor

These tables track the actual creation of the medicine on the factory floor.

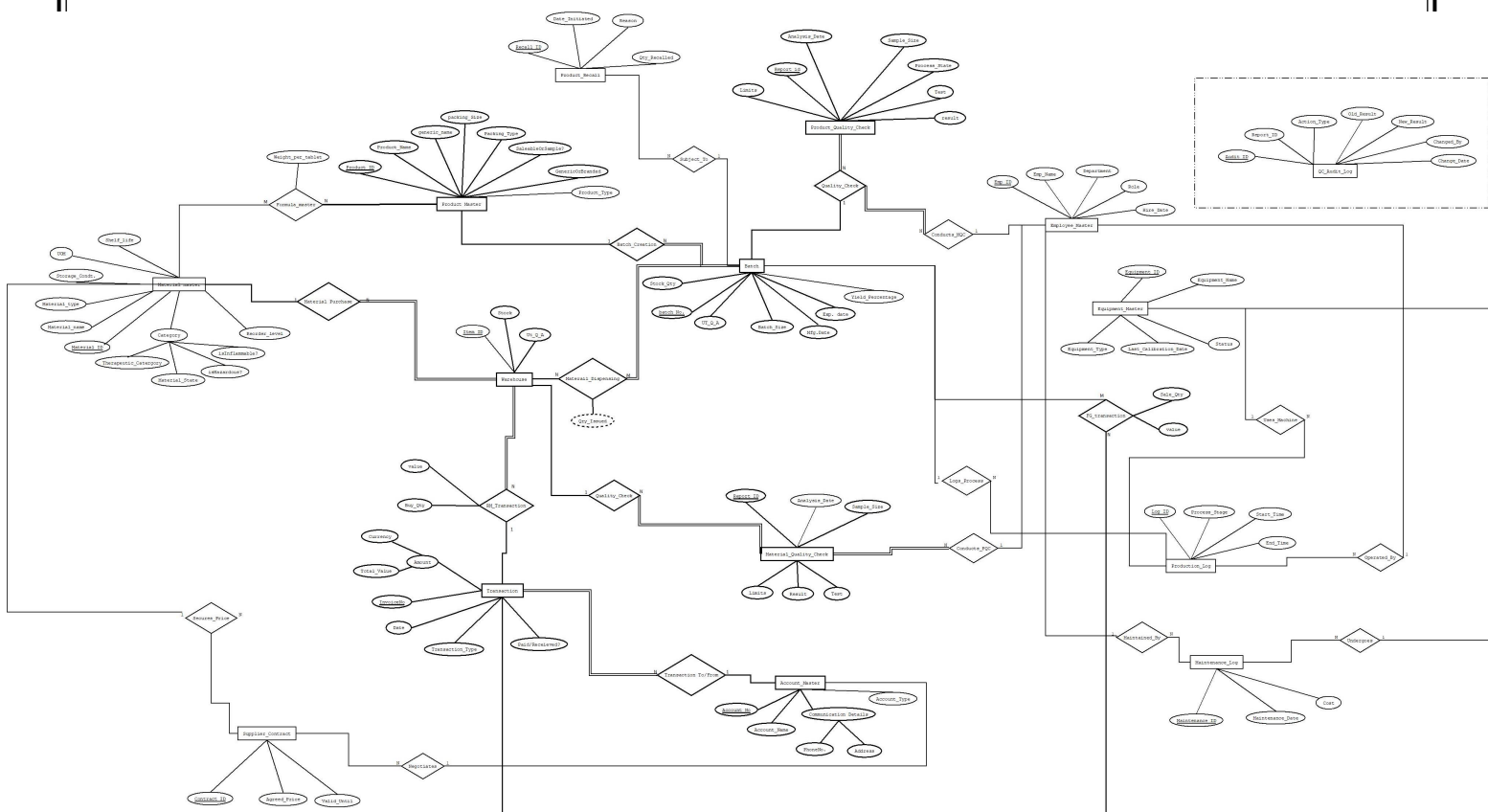
- **Formula_Master**: The exact chemical recipe (Bill of Materials) for a medicine.
Use Case: Dictates the precise ratio of raw materials required, preventing operators from mixing the wrong chemicals together.
- **Batch**: The central hub of the database; it tracks a specific production run of a medicine.
Use Case: Provides absolute traceability. It calculates the expiry date, tracks how many pills were yielded, and acts as the bridge between raw materials and final sales.
- **Material_Dispensing**: Records the exact moment a worker takes raw materials out of the warehouse to put into a batch.
Use Case: This table triggers the automated stock deduction, instantly updating warehouse inventory without requiring human math.
- **Production_Log**: A timestamped diary of machine usage featuring asynchronous clock validation.
Use Case: Allows managers to track processing bottlenecks by measuring exactly how long each manufacturing stage takes.

5. Post-Production Compliance

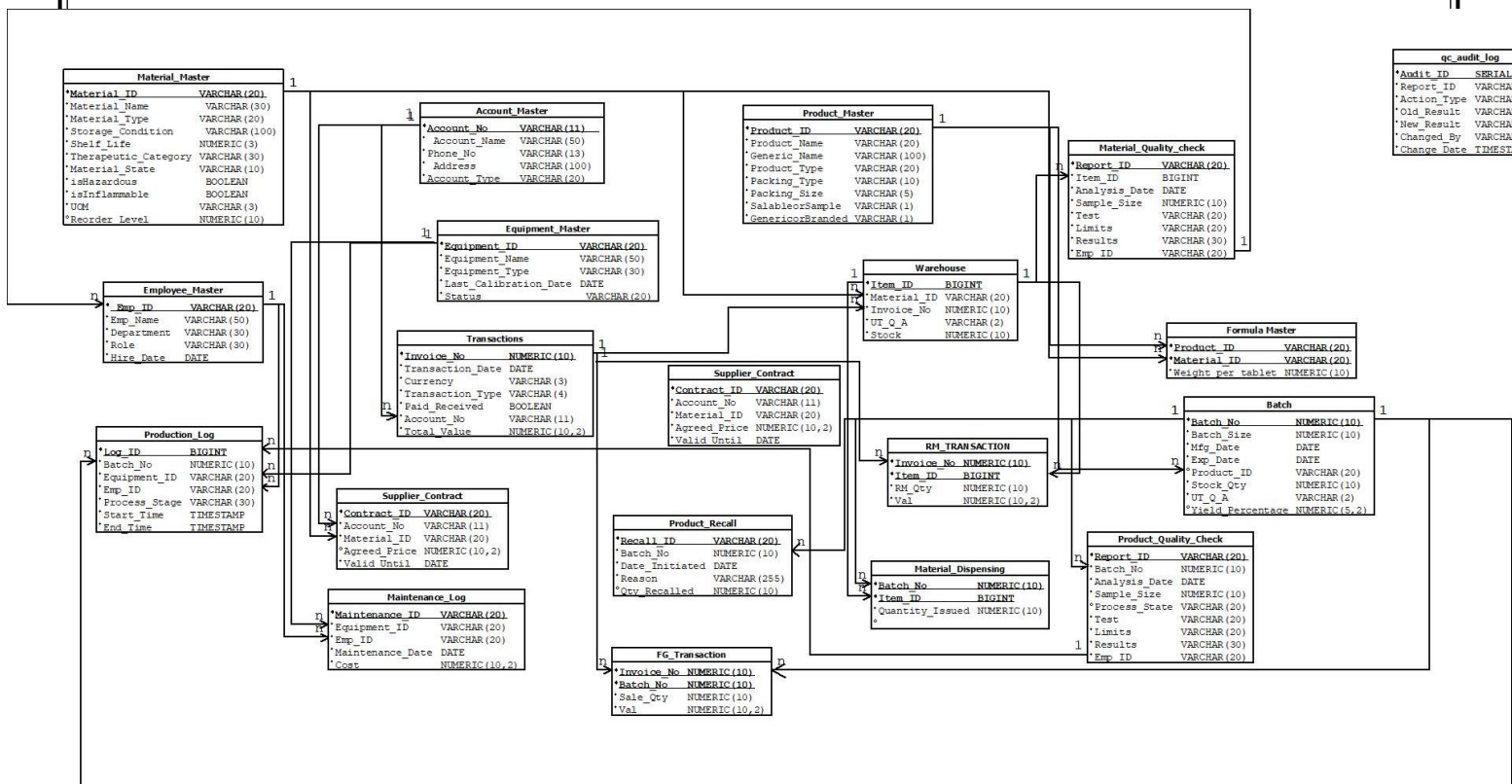
These tables enforce FDA regulations, track equipment health, and handle the medicine leaving the factory.

- **Product_Quality_Check**: The final laboratory test results for newly manufactured medicine batches.
Use Case: The ultimate gatekeeper. This table holds the PASSED/FAILED grades that determine if a medicine is legally safe to leave the factory.
- **FG_Transaction (Finished Goods)**: Records the sale of finished medicine to hospitals or distributors.
Use Case: Automatically triggers the Sales Compliance Blocker trigger. If the linked batch failed its lab test, this table rejects the sale entirely.
- **Product_Recall**: The emergency quarantine registry.
Use Case: In the event of a safety crisis, this allows executives to instantly log the recall, quarantine the lab records, and drop physical warehouse stock to zero to prevent further harm.
- **Maintenance_Log**: The historical record of machine repairs.
Use Case: Helps the finance team calculate which machines are costing the most money to keep running.

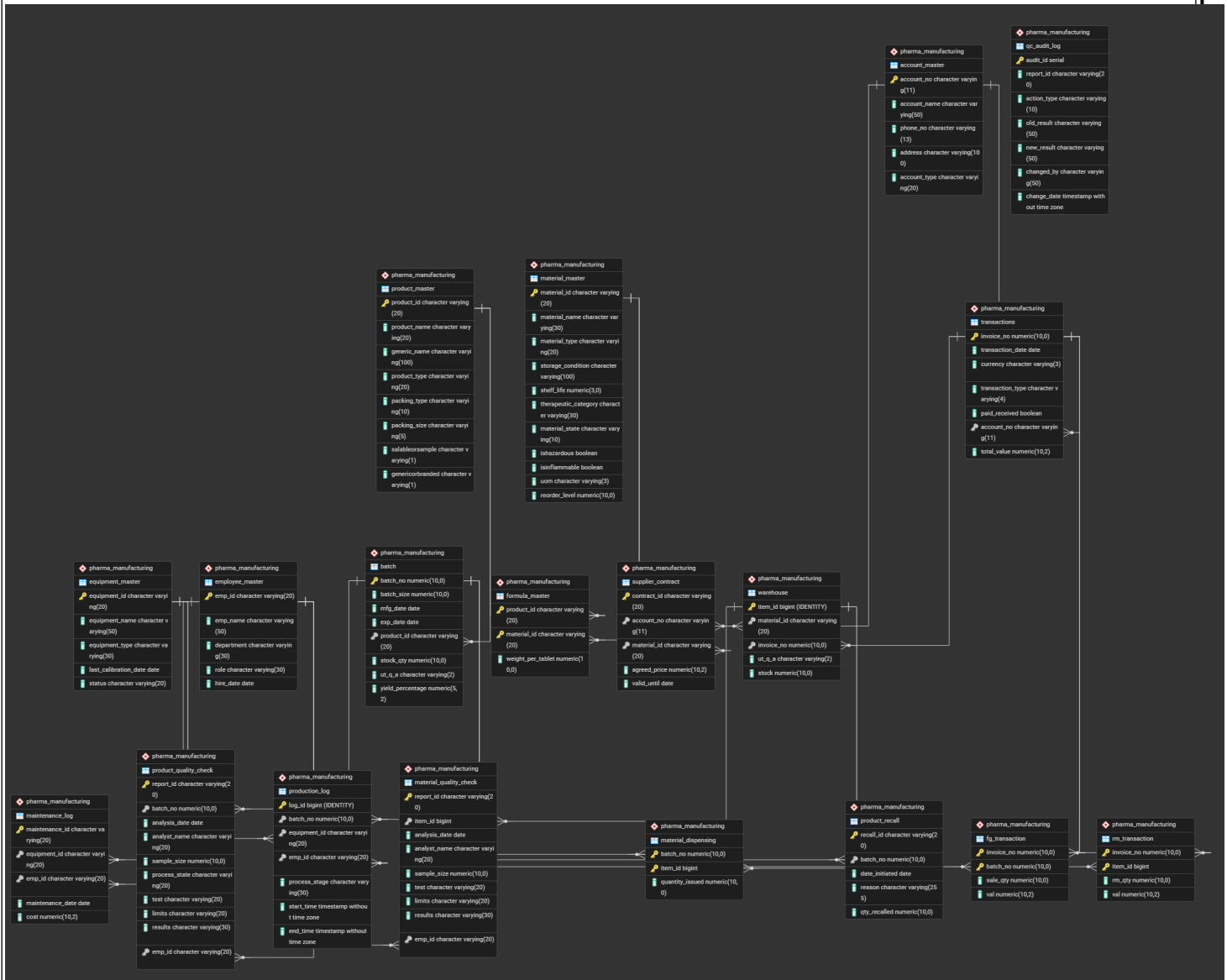
2 Entity-Relationship Diagram



3 Relational Diagram (Created using DIA)



4 Relational Diagram (Generated via PostgreSQL)



5 Normalization and Functional Dependencies

To guarantee data integrity and completely eliminate insert, update, and delete anomalies, the entire database architecture has been mathematically verified to satisfy Boyce-Codd Normal Form (BCNF). Below are the functional dependency (FD) proofs for every entity in the system.

5.1 Master Registries

5.1.1 Table: Material_Master

Let $R = \{\text{Material_ID}, \text{Material_Name}, \text{UOM}, \text{Storage_Condition}, \text{isHazardous}\}$

- (a) **Inferred FDs:** $\text{Material_ID} \rightarrow \text{Material_Name}, \text{UOM}, \text{Storage_Condition}, \text{isHazardous}$
- (b) **Minimal Set of FDs:**
 - $\text{Material_ID} \rightarrow \text{Material_Name}$
 - $\text{Material_ID} \rightarrow \text{UOM}$
 - $\text{Material_ID} \rightarrow \text{Storage_Condition}$
 - $\text{Material_ID} \rightarrow \text{isHazardous}$
- (c) **Keys:** Primary Key = $\{\text{Material_ID}\}$
- (d) **Proof of BCNF:** For every non-trivial FD ($X \rightarrow Y$), the determinant X (Material_ID) is a candidate key. No partial or transitive dependencies exist.
- (e) **Statement:** The table is in BCNF because all attributes depend strictly on the complete primary key.

5.1.2 Table: Account_Master

Let $R = \{\text{Account_No}, \text{Account_Name}, \text{Account_Type}, \text{Phone_No}, \text{Email}\}$

- (a) **Inferred FDs:** $\text{Account_No} \rightarrow \text{Account_Name}, \text{Account_Type}, \text{Phone_No}, \text{Email}$
- (b) **Minimal Set of FDs:**
 - $\text{Account_No} \rightarrow \text{Account_Name}$
 - $\text{Account_No} \rightarrow \text{Account_Type}$
 - $\text{Account_No} \rightarrow \text{Phone_No}$
 - $\text{Account_No} \rightarrow \text{Email}$
- (c) **Keys:** Primary Key = $\{\text{Account_No}\}$
- (d) **Proof of BCNF:** The only determinant in the relation is Account_No , which uniquely identifies the entire tuple.
- (e) **Statement:** The table is in BCNF as there are no dependencies on non-prime attributes.

5.1.3 Table: Employee_Master

Let $R = \{\text{Emp_ID}, \text{Emp_Name}, \text{Role}, \text{Department}\}$

- (a) **Inferred FDs:** $\text{Emp_ID} \rightarrow \text{Emp_Name}, \text{Role}, \text{Department}$
- (b) **Minimal Set of FDs:**
 $\text{Emp_ID} \rightarrow \text{Emp_Name}$
 $\text{Emp_ID} \rightarrow \text{Role}$
 $\text{Emp_ID} \rightarrow \text{Department}$
- (c) **Keys:** Primary Key = $\{\text{Emp_ID}\}$
- (d) **Proof of BCNF:** The LHS of all functional dependencies is Emp_ID , which is the superkey.
- (e) **Statement:** The table satisfies BCNF requirements perfectly.

5.1.4 Table: Equipment_Master

Let $R = \{\text{Equipment_ID}, \text{Equipment_Name}, \text{Equipment_Type}, \text{Last_Calibration}\}$

- (a) **Inferred FDs:** $\text{Equipment_ID} \rightarrow \text{Equipment_Name}, \text{Equipment_Type}, \text{Last_Calibration}$
- (b) **Minimal Set of FDs:**
 $\text{Equipment_ID} \rightarrow \text{Equipment_Name}$
 $\text{Equipment_ID} \rightarrow \text{Equipment_Type}$
 $\text{Equipment_ID} \rightarrow \text{Last_Calibration}$
- (c) **Keys:** Primary Key = $\{\text{Equipment_ID}\}$
- (d) **Proof of BCNF:** Equipment_ID is the sole determinant and acts as the unique identifier.
- (e) **Statement:** The table is in BCNF.

5.1.5 Table: Product_Master

Let $R = \{\text{Product_ID}, \text{Product_Name}, \text{Product_Type}\}$

- (a) **Inferred FDs:** $\text{Product_ID} \rightarrow \text{Product_Name}, \text{Product_Type}$
- (b) **Minimal Set of FDs:**
 $\text{Product_ID} \rightarrow \text{Product_Name}$
 $\text{Product_ID} \rightarrow \text{Product_Type}$
- (c) **Keys:** Primary Key = $\{\text{Product_ID}\}$
- (d) **Proof of BCNF:** Product_ID determines all attributes; there are no composite key partial dependencies.
- (e) **Statement:** The table is in BCNF.

5.2 Logistics and Financials

5.2.1 Table: Transactions

Let $R = \{\text{Invoice_No}, \text{Account_No}, \text{Transaction_Date}, \text{Transaction_Type}, \text{Total_Value}\}$

- (a) **Inferred FDs:** $\text{Invoice_No} \rightarrow \text{Account_No}, \text{Transaction_Date}, \text{Transaction_Type}, \text{Total_Value}$
- (b) **Minimal Set of FDs:**
 - $\text{Invoice_No} \rightarrow \text{Account_No}$
 - $\text{Invoice_No} \rightarrow \text{Transaction_Date}$
 - $\text{Invoice_No} \rightarrow \text{Transaction_Type}$
 - $\text{Invoice_No} \rightarrow \text{Total_Value}$
- (c) **Keys:** Primary Key = $\{\text{Invoice_No}\}$
- (d) **Proof of BCNF:** The invoice number uniquely identifies the transaction record. The Account_No is a foreign key but does not determine the invoice details.
- (e) **Statement:** The table is in BCNF.

5.2.2 Table: Supplier_Contract

Let $R = \{\text{Contract_ID}, \text{Account_No}, \text{Material_ID}, \text{Agreed_Price}, \text{Valid_Until}\}$

- (a) **Inferred FDs:** $\text{Contract_ID} \rightarrow \text{Account_No}, \text{Material_ID}, \text{Agreed_Price}, \text{Valid_Until}$
- (b) **Minimal Set of FDs:**
 - $\text{Contract_ID} \rightarrow \text{Account_No}$
 - $\text{Contract_ID} \rightarrow \text{Material_ID}$
 - $\text{Contract_ID} \rightarrow \text{Agreed_Price}$
 - $\text{Contract_ID} \rightarrow \text{Valid_Until}$
- (c) **Keys:** Primary Key = $\{\text{Contract_ID}\}$
- (d) **Proof of BCNF:** The synthetic key Contract_ID determines all attributes.
- (e) **Statement:** The table is mathematically in BCNF.

5.2.3 Table: Warehouse

Let $R = \{\text{Item_ID}, \text{Material_ID}, \text{Stock}, \text{Invoice_No}\}$

- (a) **Inferred FDs:** $\text{Item_ID} \rightarrow \text{Material_ID}, \text{Stock}, \text{Invoice_No}$
- (b) **Minimal Set of FDs:**
 - $\text{Item_ID} \rightarrow \text{Material_ID}$
 - $\text{Item_ID} \rightarrow \text{Stock}$
 - $\text{Item_ID} \rightarrow \text{Invoice_No}$
- (c) **Keys:** Primary Key = $\{\text{Item_ID}\}$
- (d) **Proof of BCNF:** Item_ID (representing a serialized warehouse barcode) is a superkey determining the physical location and state of the material.
- (e) **Statement:** The table successfully achieves BCNF.

5.2.4 Table: RM_Transaction

Let $R = \{\text{Invoice_No}, \text{Material_ID}, \text{Qty_Purchased}, \text{Unit_Price}\}$

- (a) **Inferred FDs:** $\{\text{Invoice_No}, \text{Material_ID}\} \rightarrow \text{Qty_Purchased}, \text{Unit_Price}$
- (b) **Minimal Set of FDs:**
 $\{\text{Invoice_No}, \text{Material_ID}\} \rightarrow \text{Qty_Purchased}$
 $\{\text{Invoice_No}, \text{Material_ID}\} \rightarrow \text{Unit_Price}$
- (c) **Keys:** Composite Primary Key = $\{\text{Invoice_No}, \text{Material_ID}\}$
- (d) **Proof of BCNF:** Neither Invoice_No nor Material_ID alone can determine the purchased quantity or price. The composite key is the only determinant.
- (e) **Statement:** Because the determinant is the full composite key, it is in BCNF.

5.3 Manufacturing Floor

5.3.1 Table: Formula_Master

Let $R = \{\text{Product_ID}, \text{Material_ID}, \text{Required_Qty}\}$

- (a) **Inferred FDs:** $\{\text{Product_ID}, \text{Material_ID}\} \rightarrow \text{Required_Qty}$
- (b) **Minimal Set of FDs:**
 $\{\text{Product_ID}, \text{Material_ID}\} \rightarrow \text{Required_Qty}$
- (c) **Keys:** Composite Primary Key = $\{\text{Product_ID}, \text{Material_ID}\}$
- (d) **Proof of BCNF:** The required quantity depends entirely on both the final product and the specific raw material ingredient.
- (e) **Statement:** The table is in BCNF.

5.3.2 Table: Batch

Let $R = \{\text{Batch_No}, \text{Batch_Size}, \text{Mfg_Date}, \text{Exp_Date}, \text{Product_ID}, \text{Stock_Qty}, \text{UT_Q_A}, \text{Yield_Perc}\}$

- (a) **Inferred FDs:** $\text{Batch_No} \rightarrow \text{Batch_Size}, \text{Mfg_Date}, \text{Exp_Date}, \text{Product_ID}, \text{Stock_Qty}, \text{UT_Q_A}, \text{Yield_Perc}$
- (b) **Minimal Set of FDs:**
 $\text{Batch_No} \rightarrow \text{Batch_Size}$
 $\text{Batch_No} \rightarrow \text{Mfg_Date}$
(Repeated for all non-prime attributes)
- (c) **Keys:** Primary Key = $\{\text{Batch_No}\}$
- (d) **Proof of BCNF:** All production metrics depend entirely on the unique Batch number.
- (e) **Statement:** The table is fully normalized to BCNF.

5.3.3 Table: Material_Dispensing

Let $R = \{\text{Batch_No}, \text{Item_ID}, \text{Quantity_Issued}\}$

- (a) **Inferred FDs:** $\{\text{Batch_No}, \text{Item_ID}\} \rightarrow \text{Quantity_Issued}$
- (b) **Minimal Set of FDs:**
 $\{\text{Batch_No}, \text{Item_ID}\} \rightarrow \text{Quantity_Issued}$
- (c) **Keys:** Composite Primary Key = $\{\text{Batch_No}, \text{Item_ID}\}$
- (d) **Proof of BCNF:** The quantity issued can only be determined by knowing both the specific batch requesting it and the specific warehouse item it is pulled from.
- (e) **Statement:** The table satisfies BCNF.

5.3.4 Table: Production_Log

Let $R = \{\text{Log_ID}, \text{Batch_No}, \text{Equipment_ID}, \text{Process_Stage}, \text{Start_Time}, \text{End_Time}\}$

- (a) **Inferred FDs:** $\text{Log_ID} \rightarrow \text{Batch_No}, \text{Equipment_ID}, \text{Process_Stage}, \text{Start_Time}, \text{End_Time}$
- (b) **Minimal Set of FDs:**
 $\text{Log_ID} \rightarrow \text{Batch_No}$
 $\text{Log_ID} \rightarrow \text{Equipment_ID}$
(Repeated for all non-prime attributes)
- (c) **Keys:** Primary Key = $\{\text{Log_ID}\}$
- (d) **Proof of BCNF:** Log_ID is the superkey. No partial dependencies on Batch_No or Equipment_ID exist.
- (e) **Statement:** The table is in BCNF.

5.4 Quality Assurance and Compliance

5.4.1 Table: Material_Quality_Check

Let $R = \{\text{Report_ID}, \text{Item_ID}, \text{Emp_ID}, \text{Analysis_Date}, \text{Results}\}$

- (a) **Inferred FDs:** $\text{Report_ID} \rightarrow \text{Item_ID}, \text{Emp_ID}, \text{Analysis_Date}, \text{Results}$
- (b) **Minimal Set of FDs:**
 $\text{Report_ID} \rightarrow \text{Item_ID}$
 $\text{Report_ID} \rightarrow \text{Emp_ID}$
(Repeated for all non-prime attributes)
- (c) **Keys:** Primary Key = $\{\text{Report_ID}\}$
- (d) **Proof of BCNF:** Report_ID uniquely identifies the specific laboratory test event.
- (e) **Statement:** The table is in BCNF.

5.4.2 Table: Product_Quality_Check

Let $R = \{\text{Report_ID}, \text{Batch_No}, \text{Emp_ID}, \text{Analysis_Date}, \text{Test}, \text{Limits}, \text{Results}\}$

- (a) **Inferred FDs:** $\text{Report_ID} \rightarrow \text{Batch_No}, \text{Emp_ID}, \text{Analysis_Date}, \text{Test}, \text{Limits}, \text{Results}$
- (b) **Minimal Set of FDs:**
 $\text{Report_ID} \rightarrow \text{Batch_No}$
 $\text{Report_ID} \rightarrow \text{Results}$
(Repeated for all non-prime attributes)
- (c) **Keys:** Primary Key = $\{\text{Report_ID}\}$
- (d) **Proof of BCNF:** The LHS of all FDs is the superkey Report_ID .
- (e) **Statement:** The table is in BCNF.

5.4.3 Table: QC_Audit_Log

Let $R = \{\text{Audit_ID}, \text{Report_ID}, \text{Action_Type}, \text{Old_Result}, \text{New_Result}, \text{Changed_By}, \text{Change_Date}\}$

- (a) **Inferred FDs:** $\text{Audit_ID} \rightarrow \text{Report_ID}, \text{Action_Type}, \text{Old_Result}, \text{New_Result}, \text{Changed_By}, \text{Change_Date}$
- (b) **Minimal Set of FDs:**
 $\text{Audit_ID} \rightarrow \text{Report_ID}$
 $\text{Audit_ID} \rightarrow \text{Action_Type}$
(Repeated for all non-prime attributes)
- (c) **Keys:** Primary Key = $\{\text{Audit_ID}\}$
- (d) **Proof of BCNF:** Audit_ID is an auto-incrementing surrogate key that determines all audit trail metrics.
- (e) **Statement:** The table is in BCNF.

5.4.4 Table: Maintenance_Log

Let $R = \{\text{Maintenance_ID}, \text{Equipment_ID}, \text{Cost}, \text{Description}, \text{Maintenance_Date}\}$

- (a) **Inferred FDs:** $\text{Maintenance_ID} \rightarrow \text{Equipment_ID}, \text{Cost}, \text{Description}, \text{Maintenance_Date}$
- (b) **Minimal Set of FDs:**
 $\text{Maintenance_ID} \rightarrow \text{Equipment_ID}$
 $\text{Maintenance_ID} \rightarrow \text{Cost}$
(Repeated for all non-prime attributes)
- (c) **Keys:** Primary Key = $\{\text{Maintenance_ID}\}$
- (d) **Proof of BCNF:** The primary key is the sole determinant.
- (e) **Statement:** The table is in BCNF.

5.5 Commercial Operations

5.5.1 Table: FG_Transaction

Let $R = \{\text{Transaction_ID}, \text{Invoice_No}, \text{Batch_No}, \text{Sale_Qty}, \text{Val}\}$

- (a) **Inferred FDs:** $\text{Transaction_ID} \rightarrow \text{Invoice_No}, \text{Batch_No}, \text{Sale_Qty}, \text{Val}$
- (b) **Minimal Set of FDs:**
 - $\text{Transaction_ID} \rightarrow \text{Invoice_No}$
 - $\text{Transaction_ID} \rightarrow \text{Batch_No}$
 - $\text{Transaction_ID} \rightarrow \text{Sale_Qty}$
 - $\text{Transaction_ID} \rightarrow \text{Val}$
- (c) **Keys:** Primary Key = $\{\text{Transaction_ID}\}$
- (d) **Proof of BCNF:** A specific line-item sale is determined exclusively by the Transaction_ID.
- (e) **Statement:** The table is in BCNF.

5.5.2 Table: Product_Recall

Let $R = \{\text{Recall_ID}, \text{Batch_No}, \text{Date_Initiated}, \text{Reason}, \text{Qty_Recalled}\}$

- (a) **Inferred FDs:** $\text{Recall_ID} \rightarrow \text{Batch_No}, \text{Date_Initiated}, \text{Reason}, \text{Qty_Recalled}$
- (b) **Minimal Set of FDs:**
 - $\text{Recall_ID} \rightarrow \text{Batch_No}$
 - $\text{Recall_ID} \rightarrow \text{Date_Initiated}$
 - $\text{Recall_ID} \rightarrow \text{Reason}$
 - $\text{Recall_ID} \rightarrow \text{Qty_Recalled}$
- (c) **Keys:** Primary Key = $\{\text{Recall_ID}\}$
- (d) **Proof of BCNF:** Recall_ID is the singular superkey determining the parameters of the emergency event.
- (e) **Statement:** The table is perfectly normalized to BCNF.

6 Data Definition Language (DDL) Scripts

The following SQL script defines the physical schema, data types, primary keys, foreign key constraints (with cascading rules), and domain constraints (CHECK clauses) for the entire PostgreSQL database.

Listing 1: Complete DDL Script for PharmaChain-DB Architecture

```
1 CREATE SCHEMA IF NOT EXISTS pharma_manufacturing;
2
3 SET search_path TO pharma_manufacturing;
4
5 -- 1. Master Tables
6 CREATE TABLE Material_Master (
7     Material_ID          VARCHAR(20)    PRIMARY KEY,
8     Material_Name        VARCHAR(30)    NOT NULL,
9     Material_Type        VARCHAR(20)    NOT NULL,
10    Storage_Condition     VARCHAR(100)   NOT NULL,
11    Shelf_Life            NUMERIC(3)     NOT NULL CHECK (
12        Shelf_Life > 0),
13    Therapeutic_Category  VARCHAR(30)    NOT NULL,
14    Material_State        VARCHAR(10)    NOT NULL,
15    isHazardous           BOOLEAN        NOT NULL,
16    isInflammable        BOOLEAN        NOT NULL,
17    UOM                  VARCHAR(3)      NOT NULL,
18    Reorder_Level        NUMERIC(10)     DEFAULT 1000 CHECK (
19        Reorder_Level > 0)
20 );
21
22 CREATE TABLE Account_Master (
23     Account_No          VARCHAR(11)     PRIMARY KEY,
24     Account_Name        VARCHAR(50)     NOT NULL,
25     Phone_No           VARCHAR(13)     NOT NULL,
26     Address            VARCHAR(100)    NOT NULL,
27     Account_Type       VARCHAR(20)     DEFAULT 'Distributor' CHECK (
28         Account_Type IN ('Supplier', 'Distributor', 'Hospital'))
29 );
30
31 CREATE TABLE Employee_Master (
32     Emp_ID             VARCHAR(20)     PRIMARY KEY,
33     Emp_Name          VARCHAR(50)     NOT NULL,
34     Department        VARCHAR(30)     NOT NULL,
35     Role              VARCHAR(30)     NOT NULL,
36     Hire_Date         DATE NOT NULL
37 );
38
39 CREATE TABLE Equipment_Master (
40     Equipment_ID       VARCHAR(20)     PRIMARY KEY,
41     Equipment_Name     VARCHAR(50)     NOT NULL,
42     Equipment_Type     VARCHAR(30)     NOT NULL,
43     Last_Calibration_Date DATE NOT NULL,
44     Status             VARCHAR(20)     NOT NULL CHECK (Status IN
```

```

        ('Active', 'Maintenance'))
42 );
43
44 CREATE TABLE Product_Master (
45     Product_ID        VARCHAR(20) PRIMARY KEY,
46     Product_Name      VARCHAR(20) NOT NULL,
47     Generic_Name      VARCHAR(100) NOT NULL,
48     Product_Type      VARCHAR(20) NOT NULL,
49     Packing_Type      VARCHAR(10) NOT NULL,
50     Packing_Size      VARCHAR(5)  NOT NULL,
51     SalableorSample   VARCHAR(1)  NOT NULL CHECK (
        SalableorSample IN ('M','S')),
52     GenericorBranded  VARCHAR(1)  NOT NULL CHECK (
        GenericorBranded IN ('G','B'))
53 );
54
55 -- 2. Transaction & Contract Tables
56 CREATE TABLE Transactions (
57     Invoice_No         NUMERIC(10)   PRIMARY KEY,
58     Transaction_Date   DATE          NOT NULL,
59     Currency          VARCHAR(3)     NOT NULL,
60     Transaction_Type   VARCHAR(4)     NOT NULL CHECK (
        Transaction_Type IN ('buy','sell')),
61     Paid_Received     BOOLEAN        NOT NULL,
62     Account_No        VARCHAR(11)    REFERENCES Account_Master(
        Account_No) ON DELETE CASCADE ON UPDATE CASCADE,
63     Total_Value       NUMERIC(10,2)  NOT NULL CHECK (Total_Value
        > 0)
64 );
65
66 CREATE TABLE Supplier_Contract (
67     Contract_ID       VARCHAR(20) PRIMARY KEY,
68     Account_No        VARCHAR(11) NOT NULL REFERENCES
        Account_Master(Account_No) ON DELETE CASCADE ON UPDATE
        CASCADE,
69     Material_ID       VARCHAR(20) NOT NULL REFERENCES
        Material_Master(Material_ID) ON DELETE CASCADE ON UPDATE
        CASCADE,
70     Agreed_Price      NUMERIC(10,2) NOT NULL CHECK (Agreed_Price >
        0),
71     Valid_Until      DATE NOT NULL
72 );
73
74 -- 3. Warehouse & Inventory
75 CREATE TABLE Warehouse (
76     Item_ID          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY
77     ,
78     Material_ID      VARCHAR(20) NOT NULL REFERENCES Material_Master
        (Material_ID) ON DELETE CASCADE ON UPDATE CASCADE,
79     Invoice_No        NUMERIC(10) NOT NULL REFERENCES Transactions(
        Invoice_No) ON DELETE CASCADE ON UPDATE CASCADE,

```

```

79      UT_Q_A          VARCHAR(2)   NOT NULL ,
80      Stock           NUMERIC(10)  NOT NULL CHECK (Stock > 0),
81      CONSTRAINT uq_warehouse_mat_inv UNIQUE (Material_ID,
      Invoice_No)
82 );
83
84 CREATE TABLE RM_Transaction (
85     Invoice_No NUMERIC(10)   NOT NULL REFERENCES Transactions(
      Invoice_No) ON DELETE CASCADE ON UPDATE CASCADE ,
86     Item_ID    BIGINT       NOT NULL REFERENCES Warehouse(
      Item_ID) ON DELETE CASCADE ON UPDATE CASCADE ,
87     RM_Qty     NUMERIC(10)   NOT NULL CHECK (RM_Qty > 0),
88     Val        NUMERIC(10,2) NOT NULL CHECK (Val > 0),
89     CONSTRAINT pk_rm_transaction PRIMARY KEY (Invoice_No,
      Item_ID)
90 );
91
92 CREATE TABLE Material_Quality_Check (
93     Report_ID    VARCHAR(20)   PRIMARY KEY ,
94     Item_ID      BIGINT        NOT NULL ,
95     Analysis_Date DATE         NOT NULL ,
96     Analyst_Name VARCHAR(20)   NOT NULL ,
97     Sample_Size  NUMERIC(10)   NOT NULL CHECK (Sample_Size > 0)
98     ,
99     Test         VARCHAR(20)   NOT NULL ,
100    Limits        VARCHAR(20)   NOT NULL ,
101    Results       VARCHAR(30)   NOT NULL ,
102    Emp_ID        VARCHAR(20)   REFERENCES Employee_Master(
      Emp_ID) ON DELETE SET NULL ,
103    CONSTRAINT fk_mqc_warehouse FOREIGN KEY (Item_ID)
      REFERENCES Warehouse(Item_ID) ON DELETE CASCADE ON
      UPDATE CASCADE
104 );
105
106 -- 4. Manufacturing & Production
107 CREATE TABLE Formula_Master (
108     Product_ID    VARCHAR(20)  NOT NULL REFERENCES
      Product_Master(Product_ID) ON DELETE CASCADE ON UPDATE
      CASCADE ,
109     Material_ID   VARCHAR(20)  NOT NULL REFERENCES
      Material_Master(Material_ID) ON DELETE CASCADE ON UPDATE
      CASCADE ,
110     Weight_per_tablet NUMERIC(10) NOT NULL CHECK (
      Weight_per_tablet > 0),
111     CONSTRAINT pk_formula_master PRIMARY KEY (Product_ID,
      Material_ID)
112 );
113
114 CREATE TABLE Batch (
115     Batch_No      NUMERIC(10)   PRIMARY KEY ,
116     Batch_Size    NUMERIC(10)   NOT NULL CHECK (Batch_Size >

```

```

    0),
116     Mfg_Date          DATE          NOT NULL,
117     Exp_Date          DATE,
118     Product_ID        VARCHAR(20) REFERENCES Product_Master(
        Product_ID) ON DELETE CASCADE ON UPDATE CASCADE,
119     Stock_Qty         NUMERIC(10) NOT NULL CHECK (Stock_Qty >=
        0),
120     UT_Q_A            VARCHAR(2)  NOT NULL,
121     Yield_Percentage  NUMERIC(5,2) DEFAULT 98.50 CHECK (
        Yield_Percentage >= 0 AND Yield_Percentage <= 100)
122 );
123
124 CREATE TABLE Material_Dispensing (
125     Batch_No          NUMERIC(10) NOT NULL REFERENCES Batch(
        Batch_No) ON DELETE CASCADE ON UPDATE CASCADE,
126     Item_ID           BIGINT      NOT NULL REFERENCES Warehouse(
        Item_ID) ON DELETE CASCADE ON UPDATE CASCADE,
127     Quantity_Issued   NUMERIC(10) NOT NULL CHECK (
        Quantity_Issued > 0),
128     CONSTRAINT pk_material_dispensing PRIMARY KEY (Batch_No,
        Item_ID)
129 );
130
131 CREATE TABLE Production_Log (
132     Log_ID            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY
        KEY,
133     Batch_No          NUMERIC(10) NOT NULL REFERENCES Batch(
        Batch_No) ON DELETE CASCADE ON UPDATE CASCADE,
134     Equipment_ID      VARCHAR(20) REFERENCES Equipment_Master(
        Equipment_ID) ON DELETE SET NULL ON UPDATE CASCADE,
135     Emp_ID            VARCHAR(20) REFERENCES Employee_Master(Emp_ID
        ) ON DELETE SET NULL ON UPDATE CASCADE,
136     Process_Stage     VARCHAR(30) NOT NULL,
137     Start_Time        TIMESTAMP NOT NULL,
138     End_Time          TIMESTAMP NOT NULL,
139     CONSTRAINT chk_time CHECK (End_Time > Start_Time)
140 );
141
142 -- 5. Final Quality & Post-Production
143 CREATE TABLE Product_Quality_Check (
144     Report_ID         VARCHAR(20) PRIMARY KEY,
145     Batch_No          NUMERIC(10) REFERENCES Batch(Batch_No) ON
        DELETE CASCADE ON UPDATE CASCADE,
146     Analysis_Date     DATE          NOT NULL,
147     Analyst_Name      VARCHAR(20)  NOT NULL,
148     Sample_Size       NUMERIC(10)  NOT NULL CHECK (Sample_Size > 0)
        ,
149     Process_State     VARCHAR(20)  NOT NULL,
150     Test              VARCHAR(20)  NOT NULL,
151     Limits            VARCHAR(20)  NOT NULL,
152     Results           VARCHAR(30)  NOT NULL,

```

```

153     Emp_ID          VARCHAR(20) REFERENCES Employee_Master(
154         Emp_ID) ON DELETE SET NULL
155 );
156 CREATE TABLE FG_Transaction (
157     Invoice_No NUMERIC(10) NOT NULL REFERENCES Transactions(
158         Invoice_No) ON DELETE CASCADE ON UPDATE CASCADE,
159     Batch_No   NUMERIC(10) NOT NULL REFERENCES Batch(Batch_No
160         ) ON DELETE CASCADE ON UPDATE CASCADE,
161     Sale_Qty   NUMERIC(10) NOT NULL CHECK (Sale_Qty > 0),
162     Val        NUMERIC(10,2) NOT NULL CHECK (Val > 0),
163     CONSTRAINT pk_fg_transaction PRIMARY KEY (Invoice_No,
164         Batch_No)
165 );
166 CREATE TABLE Product_Recall (
167     Recall_ID      VARCHAR(20) PRIMARY KEY,
168     Batch_No       NUMERIC(10) NOT NULL REFERENCES Batch(
169         Batch_No) ON DELETE CASCADE ON UPDATE CASCADE,
170     Date_Initiated DATE NOT NULL,
171     Reason          VARCHAR(255) NOT NULL,
172     Qty_Recalled    NUMERIC(10) NOT NULL CHECK (Qty_Recalled >
173         0)
174 );
175 CREATE TABLE Maintenance_Log (
176     Maintenance_ID VARCHAR(20) PRIMARY KEY,
177     Equipment_ID    VARCHAR(20) NOT NULL REFERENCES
178         Equipment_Master(Equipment_ID) ON DELETE CASCADE ON
179         UPDATE CASCADE,
180     Emp_ID          VARCHAR(20) REFERENCES Employee_Master(
181         Emp_ID) ON DELETE SET NULL ON UPDATE CASCADE,
182     Maintenance_Date DATE NOT NULL,
183     Cost            NUMERIC(10,2) NOT NULL CHECK (Cost >= 0)
184 );
185 -- TABLE: Quality Control Audit Log (The Hidden Vault)
186 CREATE TABLE QC_Audit_Log (
187     Audit_ID SERIAL PRIMARY KEY,
188     Report_ID VARCHAR(20),
189     Action_Type VARCHAR(10),
190     Old_Result VARCHAR(50),
191     New_Result VARCHAR(50),
192     Changed_By VARCHAR(50),
193     Change_Date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
194 );

```

7 Data Manipulation Language (DML) Scripts

The following script populates the PharmaChain-DB schema with synthetic enterprise data, simulating a fully operational factory environment. It establishes the master registries, physical inventory, multi-currency financial transactions, and highly inter-connected production batch logs.

Listing 2: Enterprise Data Insertion (DML) Script

```
1 INSERT INTO Employee_Master (Emp_ID, Emp_Name, Department, Role
   , Hire_Date) VALUES
2 ('EMP001', 'Dr. Ajay Sharma', 'Quality_Control', 'Lead_Analyst',
   , '2015-04-12'),
3 ('EMP002', 'Hitesh Patel', 'Quality_Control', 'Lab_Technician',
   , '2018-01-10'),
4 ('EMP003', 'Sunil Desai', 'Production', 'Machine_Operator', '
   2017-06-22'),
5 ('EMP004', 'Vikram Singh', 'Production', 'Machine_Operator', '
   2019-03-15'),
6 ('EMP005', 'Anita Roy', 'Production', 'Floor_Supervisor', '
   2014-11-05'),
7 ('EMP006', 'Ramesh Kumar', 'Maintenance', 'Senior_Mechanic', '
   2016-08-19'),
8 ('EMP007', 'Suresh Pillai', 'Maintenance', 'Electrical_Engineer
   ', '2020-02-01'),
9 ('EMP008', 'Kavita Menon', 'Supply_Chain', 'Procurement_Officer
   ', '2018-09-11'),
10 ('EMP009', 'Nitin Gadkari', 'Warehouse', 'Inventory_Manager', '
   2015-07-07'),
11 ('EMP010', 'Dr. Meera Iyer', 'Quality_Assurance', 'QA_Director'
   , '2012-05-30'),
12 ('EMP011', 'Rahul Verma', 'Quality_Control', 'Analyst', '
   2021-03-10'),
13 ('EMP012', 'Priya Singh', 'Production', 'Packaging_Specialist',
   , '2020-07-15'),
14 ('EMP013', 'Amit Shah', 'Warehouse', 'Forklift_Operator', '
   2019-11-20'),
15 ('EMP014', 'Neha Gupta', 'Quality_Assurance', 'Compliance_
   Officer', '2016-02-14'),
16 ('EMP015', 'Sanjay Dutt', 'Maintenance', 'Plumbing_Tech', '
   2018-05-05'),
17 ('EMP016', 'Pooja Reddy', 'Supply_Chain', 'Vendor_Liaison', '
   2022-01-10'),
18 ('EMP017', 'Arjun Nair', 'Production', 'Machine_Operator', '
   2021-08-25'),
19 ('EMP018', 'Karan Johar', 'Warehouse', 'Logistics_Clerk', '
   2020-12-01'),
20 ('EMP019', 'Riya Sen', 'Quality_Control', 'Microbiologist', '
   2017-09-15'),
21 ('EMP020', 'Manoj Bajpayee', 'Production', 'Shift_Manager', '
   2013-04-20');
```

```

23 INSERT INTO Equipment_Master (Equipment_ID, Equipment_Name,
    Equipment_Type, Last_Calibration_Date, Status) VALUES
24 ('EQ001', 'High_Shear_Mixer_A', 'Mixer', '2023-01-15', 'Active'
    ),
25 ('EQ002', 'High_Shear_Mixer_B', 'Mixer', '2023-01-20', 'Active'
    ),
26 ('EQ003', 'Fluid_Bed_Granulator_1', 'Granulator', '2023-01-10',
    'Maintenance'),
27 ('EQ004', 'Rotary_Tablet_Press_1', 'Compressor', '2023-01-05',
    'Active'),
28 ('EQ005', 'Rotary_Tablet_Press_2', 'Compressor', '2023-01-12',
    'Active'),
29 ('EQ006', 'Auto_Film_Coater', 'Coater', '2023-01-01', 'Active')
    ,
30 ('EQ007', 'Blister_Packaging_Line_A', 'Packaging', '2023-01-28',
    'Active'),
31 ('EQ008', 'Bottle_Filling_Line_B', 'Packaging', '2023-01-15', '
    Active'),
32 ('EQ009', 'Liquid_Mixing_Tank_500L', 'Liquid_Processor', '
    2023-01-10', 'Active'),
33 ('EQ010', 'Syrup_Bottling_Station', 'Packaging', '2023-01-01',
    'Active'),
34 ('EQ011', 'Capsule_Filling_Machine', 'Filler', '2023-01-11', '
    Active'),
35 ('EQ012', 'V-Cone_Blender', 'Mixer', '2023-01-09', 'Active'),
36 ('EQ013', 'Fluid_Bed_Granulator_2', 'Granulator', '2023-01-18',
    'Active'),
37 ('EQ014', 'Tablet_Checking_Machine', 'QA_Equipment', '
    2023-01-22', 'Active'),
38 ('EQ015', 'Cartoning_Machine', 'Packaging', '2023-01-30', '
    Maintenance'),
39 ('EQ016', 'Purified_Water_System', 'Utility', '2023-01-02', '
    Active'),
40 ('EQ017', 'Air_Handling_Unit', 'Utility', '2023-01-03', 'Active
    '),
41 ('EQ018', 'Industrial_Autoclave', 'Sterilizer', '2023-01-14', '
    Active'),
42 ('EQ019', 'Metal_Detector_Line', 'QA_Equipment', '2023-01-25',
    'Active'),
43 ('EQ020', 'Shrink_Wrapping_Machine', 'Packaging', '2023-01-29',
    'Active');
44
45 INSERT INTO Account_Master (Account_No, Account_Name, Phone_No,
    Address, Account_Type) VALUES
46 ('ACC001', 'AK_Pharma_Raw_Materials', '911023456789', 'Vatva,
    Gujarat', 'Supplier'),
47 ('ACC002', 'Stallion_Chemical_Corp', '919099234567', 'Changodar
    , Gujarat', 'Supplier'),
48 ('ACC003', 'West_Coast_API_Labs', '913243423342', 'Business
    Park, Ahmedabad', 'Supplier'),
49 ('ACC004', 'Sagar_Bulk_Drugs', '913891819333', 'Pharma_Hub,

```

```

50      Mumbai', 'Supplier'),
51      ('ACC005', 'Sehat-E-Zafran_Labs', '651231312312', 'ADNEC, Abu_
      Dhabi', 'Supplier'),
52      ('ACC006', 'Justeen_Excipients', '233312311333', 'Kosad_Road,
      Nigeria', 'Supplier'),
53      ('ACC007', 'Global_Chemicals_Ltd', '912233445566', 'Navi_Mumbai
      ', 'Supplier'),
54      ('ACC008', 'Pure_APIs_Inc', '18005551234', 'New_York, USA', '
      Supplier'),
55      ('ACC009', 'EuroPharma_Raw', '442071234567', 'London, UK', '
      Supplier'),
56      ('ACC010', 'Zhengzhou_Organics', '861012345678', 'Zhengzhou,
      China', 'Supplier'),
57      ('ACC011', 'Zydus_Hospitals', '915685685887', 'Ahmedabad', '
      Hospital'),
58      ('ACC012', 'Apollo_Pharmacy_Chain', '918565655675', 'Ahmedabad'
      , 'Distributor'),
59      ('ACC013', 'Silis_Global_Distributors', '311312312312', 'Jaipur
      ', 'Distributor'),
60      ('ACC014', 'Danadams_Africa_Supply', '651308080809', 'Lagos', '
      Distributor'),
61      ('ACC015', 'Fortis_Healthcare', '911122334455', 'Delhi', '
      Hospital'),
62      ('ACC016', 'MedPlus_Network', '919988776655', 'Hyderabad', '
      Distributor'),
63      ('ACC017', 'Max_Super_Speciality', '917766554433', 'Gurgaon', '
      Hospital'),
64      ('ACC018', 'NetMeds_Delivery', '918877665544', 'Chennai', '
      Distributor'),
65      ('ACC019', 'Pharmeasy_Logistics', '916655443322', 'Mumbai', '
      Distributor'),
66      ('ACC020', 'AIIMS_Central', '911123456789', 'New_Delhi', '
      Hospital');
67
68      INSERT INTO Material_Master (Material_ID, Material_Name,
69      Material_Type, Storage_Condition, Shelf_Life,
70      Therapeutic_Category, Material_State, isHazardous,
71      isInflammable, UOM, Reorder_Level) VALUES
72      ('MAT001', 'Paracetamol_API', 'Drug', 'Room_Temp_<30C', 24, '
      Antipyretic', 'Solid', FALSE, FALSE, 'kg', 5000),
73      ('MAT002', 'Omeprazole', 'Drug', 'Room_Temp_<25C', 24, 'Anti-
      ulcer', 'Solid', FALSE, FALSE, 'kg', 2000),
74      ('MAT003', 'Azithromycin_Dihydrate', 'Drug', 'Room_Temp_<25C',
      12, 'Antibiotic', 'Solid', FALSE, FALSE, 'kg', 1500),
75      ('MAT004', 'Atorvastatin', 'Drug', 'Protect_from_light', 18, '
      Cardio-vascular', 'Solid', FALSE, FALSE, 'kg', 1000),
76      ('MAT005', 'Amoxycillin_Trihydrate', 'Drug', 'Store_in_dry_
      conditions', 18, 'Antibiotic', 'Solid', TRUE, FALSE, 'kg',
      3000),
77      ('MAT006', 'Diphenhydramine', 'Drug', 'Room_Temp_<25C', 12, '
      Anti-cold', 'Solid', FALSE, FALSE, 'kg', 1000),

```



```

74 ('MAT007', 'Sodium_Starch_Glycolate', 'Excipient', 'Cool_and_dry_place', 36, 'Disintegrating_Agent', 'Solid', FALSE,
    FALSE, 'kg', 8000),
75 ('MAT008', 'Methyl_Cellulose', 'Excipient', 'Cool_and_dry_place', 48, 'Binder', 'Solid', FALSE, FALSE, 'kg', 10000),
76 ('MAT009', 'Sucrose', 'Excipient', 'Cool_and_dry_place', 36, 'Sweetener', 'Solid', FALSE, FALSE, 'kg', 15000),
77 ('MAT010', 'Starch', 'Excipient', 'Cool_and_dry_place', 48, 'Filler', 'Solid', FALSE, FALSE, 'kg', 20000),
78 ('MAT011', 'Isopropyl_Alcohol', 'Excipient', 'Protect_from_light', 24, 'Solvent', 'Liquid', TRUE, TRUE, 'ltr', 5000),
79 ('MAT012', 'Vitamin_A', 'Supplement', 'Room_Temp_<_30C', 36, 'Nutritional', 'Solid', FALSE, FALSE, 'kg', 500),
80 ('MAT013', 'Vitamin_D3', 'Supplement', 'Room_Temp_<_30C', 24, 'Nutritional', 'Solid', FALSE, FALSE, 'kg', 500),
81 ('MAT014', 'Lactose_Monohydrate', 'Excipient', 'Dry_place', 48, 'Diluent', 'Solid', FALSE, FALSE, 'kg', 5000),
82 ('MAT015', 'Magnesium_Stearate', 'Excipient', 'Dry_place', 36, 'Lubricant', 'Solid', FALSE, FALSE, 'kg', 2000),
83 ('MAT016', 'Talc', 'Excipient', 'Dry_place', 60, 'Glidant', 'Solid', FALSE, FALSE, 'kg', 3000),
84 ('MAT017', 'Ciprofloxacin', 'Drug', 'Protect_from_light', 24, 'Antibiotic', 'Solid', FALSE, FALSE, 'kg', 1500),
85 ('MAT018', 'Ibuprofen', 'Drug', 'Room_Temp_<_25C', 36, 'Analgesic', 'Solid', FALSE, FALSE, 'kg', 4000),
86 ('MAT019', 'Glimepiride', 'Drug', 'Room_Temp_<_25C', 24, 'Anti-diabetic', 'Solid', FALSE, FALSE, 'kg', 1000),
87 ('MAT020', 'Propylene_Glycol', 'Excipient', 'Room_Temp_<_25C', 24, 'Solvent', 'Liquid', FALSE, TRUE, 'ltr', 2500);
88
89 INSERT INTO Product_Master (Product_ID, Product_Name,
    Generic_Name, Product_Type, Packing_Type, Packing_Size,
    SalableorSample, GenericorBranded) VALUES
90 ('PRD001', 'Parabufen', 'Paracetamol_Ibuprofen', 'Tablet', 'BLI', '3x10', 'M', 'B'),
91 ('PRD002', 'Omsergy', 'Omeprazole', 'Capsule', 'ALU', '10x10', 'M', 'B'),
92 ('PRD003', 'Exicof', 'Cough_Syrup', 'Syrup', 'BOT', '1x1', 'M', 'B'),
93 ('PRD004', 'Atorvastatin_20', 'Atorvastatin', 'Tablet', 'ALU', '3x10', 'M', 'G'),
94 ('PRD005', 'Amoxyn', 'Amoxycillin_Oral_Susp', 'Syrup', 'BOT', '1x1', 'M', 'B'),
95 ('PRD006', 'Vitarich', 'Multivitamin', 'Tablet', 'ALU', '2x10', 'M', 'B'),
96 ('PRD007', 'Zithrol_500', 'Azithromycin', 'Tablet', 'BLI', '1x3', 'M', 'B'),
97 ('PRD008', 'Aller-D', 'Diphenhydramine', 'Capsule', 'BLI', '2x10', 'M', 'B'),
98 ('PRD009', 'Cipro-Max', 'Ciprofloxacin', 'Tablet', 'ALU', '10x10', 'M', 'B'),

```

```

99 ('PRD010', 'Ibupro-G', 'Ibuprofen', 'Tablet', 'BLI', '5x10', 'M
    ', 'G'),
100 ('PRD011', 'Glimeta_2', 'Glimepiride', 'Tablet', 'ALU', '3x15',
    'M', 'B'),
101 ('PRD012', 'Para-500', 'Paracetamol', 'Tablet', 'BLI', '10x10',
    'S', 'G'),
102 ('PRD013', 'Para-650', 'Paracetamol', 'Tablet', 'BLI', '10x10',
    'M', 'G'),
103 ('PRD014', 'Vit-D_Core', 'Vitamin_D3', 'Capsule', 'BOT', '1x30'
    , 'M', 'B'),
104 ('PRD015', 'Ome-Protect', 'Omeprazole', 'Capsule', 'ALU', '2x15'
    , 'S', 'B'),
105 ('PRD016', 'Amox-500', 'Amoxycillin', 'Capsule', 'BLI', '3x10',
    'M', 'G'),
106 ('PRD017', 'Atorva-10', 'Atorvastatin', 'Tablet', 'ALU', '3x10'
    , 'S', 'G'),
107 ('PRD018', 'Zithro-Susp', 'Azithromycin', 'Syrup', 'BOT', '1x1'
    , 'M', 'B'),
108 ('PRD019', 'Glime-M', 'Glimepiride', 'Tablet', 'ALU', '5x10', '
    M', 'G'),
109 ('PRD020', 'Cipro-Drops', 'Ciprofloxacin', 'Liquid', 'BOT', '1
    x1', 'M', 'B');
110
111 INSERT INTO Transactions (Invoice_No, Transaction_Date,
    Currency, Transaction_Type, Paid_Received, Account_No,
    Total_Value) VALUES
112 (1001, '2023-02-01', 'INR', 'buy', TRUE, 'ACC001', 150000.00),
113 (1002, '2023-02-02', 'INR', 'buy', TRUE, 'ACC002', 120000.00),
114 (1003, '2023-02-03', 'INR', 'buy', FALSE, 'ACC003', 180000.00),
115 (1004, '2023-02-04', 'USD', 'buy', TRUE, 'ACC004', 50000.00),
116 (1005, '2023-02-05', 'AED', 'buy', TRUE, 'ACC005', 75000.00),
117 (1006, '2023-02-06', 'INR', 'buy', TRUE, 'ACC006', 40000.00),
118 (1007, '2023-02-07', 'INR', 'buy', TRUE, 'ACC007', 95000.00),
119 (1008, '2023-02-08', 'USD', 'buy', FALSE, 'ACC008', 110000.00),
120 (1009, '2023-02-09', 'EUR', 'buy', TRUE, 'ACC009', 200000.00),
121 (1010, '2023-02-10', 'CNY', 'buy', TRUE, 'ACC010', 130000.00),
122 (1011, '2023-02-11', 'INR', 'buy', TRUE, 'ACC001', 60000.00),
123 (1012, '2023-02-12', 'INR', 'buy', TRUE, 'ACC002', 70000.00),
124 (1013, '2023-02-13', 'INR', 'buy', TRUE, 'ACC003', 80000.00),
125 (1014, '2023-02-14', 'INR', 'buy', FALSE, 'ACC004', 90000.00),
126 (1015, '2023-02-15', 'INR', 'buy', TRUE, 'ACC005', 100000.00),
127 (1016, '2023-02-16', 'INR', 'buy', TRUE, 'ACC006', 110000.00),
128 (1017, '2023-02-17', 'INR', 'buy', TRUE, 'ACC007', 120000.00),
129 (1018, '2023-02-18', 'INR', 'buy', TRUE, 'ACC008', 130000.00),
130 (1019, '2023-02-19', 'INR', 'buy', FALSE, 'ACC009', 140000.00),
131 (1020, '2023-02-20', 'INR', 'buy', TRUE, 'ACC010', 150000.00),
132 (2001, '2023-04-01', 'INR', 'sell', TRUE, 'ACC011', 250000.00),
133 (2002, '2023-04-02', 'INR', 'sell', TRUE, 'ACC012', 300000.00),
134 (2003, '2023-04-03', 'INR', 'sell', FALSE, 'ACC013', 150000.00)
    ,
135 (2004, '2023-04-04', 'INR', 'sell', TRUE, 'ACC014', 400000.00),

```

```

136 (2005, '2023-04-05', 'INR', 'sell', TRUE, 'ACC015', 500000.00),
137 (2006, '2023-04-06', 'INR', 'sell', TRUE, 'ACC016', 200000.00),
138 (2007, '2023-04-07', 'INR', 'sell', TRUE, 'ACC017', 350000.00),
139 (2008, '2023-04-08', 'INR', 'sell', FALSE, 'ACC018', 450000.00)
140 ,
141 (2009, '2023-04-09', 'INR', 'sell', TRUE, 'ACC019', 120000.00),
142 (2010, '2023-04-10', 'INR', 'sell', TRUE, 'ACC020', 280000.00),
143 (2011, '2023-04-11', 'INR', 'sell', TRUE, 'ACC011', 110000.00),
144 (2012, '2023-04-12', 'INR', 'sell', TRUE, 'ACC012', 220000.00),
145 (2013, '2023-04-13', 'INR', 'sell', TRUE, 'ACC013', 330000.00),
146 (2014, '2023-04-14', 'INR', 'sell', FALSE, 'ACC014', 440000.00)
147 ,
148 (2015, '2023-04-15', 'INR', 'sell', TRUE, 'ACC015', 550000.00),
149 (2016, '2023-04-16', 'INR', 'sell', TRUE, 'ACC016', 660000.00),
150 (2017, '2023-04-17', 'INR', 'sell', TRUE, 'ACC017', 770000.00),
151 (2018, '2023-04-18', 'INR', 'sell', FALSE, 'ACC018', 880000.00)
152 ,
153 (2019, '2023-04-19', 'INR', 'sell', TRUE, 'ACC019', 990000.00),
154 (2020, '2023-04-20', 'INR', 'sell', TRUE, 'ACC020', 1000000.00)
155 ;
156
157 INSERT INTO Supplier_Contract (Contract_ID, Account_No,
158     Material_ID, Agreed_Price, Valid_Until) VALUES
159 ('CON001', 'ACC001', 'MAT001', 15.50, '2024-12-31'),
160 ('CON002', 'ACC002', 'MAT002', 45.00, '2024-12-31'),
161 ('CON003', 'ACC003', 'MAT003', 120.00, '2025-06-30'),
162 ('CON004', 'ACC004', 'MAT004', 85.00, '2025-12-31'),
163 ('CON005', 'ACC005', 'MAT005', 30.00, '2024-12-31'),
164 ('CON006', 'ACC006', 'MAT006', 22.50, '2024-08-15'),
165 ('CON007', 'ACC007', 'MAT007', 5.00, '2025-01-01'),
166 ('CON008', 'ACC008', 'MAT008', 8.50, '2024-11-30'),
167 ('CON009', 'ACC009', 'MAT009', 2.00, '2026-12-31'),
168 ('CON010', 'ACC010', 'MAT010', 1.50, '2026-12-31'),
169 ('CON011', 'ACC001', 'MAT011', 12.00, '2024-10-31'),
170 ('CON012', 'ACC002', 'MAT012', 65.00, '2025-05-31'),
171 ('CON013', 'ACC003', 'MAT013', 55.00, '2025-05-31'),
172 ('CON014', 'ACC004', 'MAT014', 4.00, '2024-12-31'),
173 ('CON015', 'ACC005', 'MAT015', 6.00, '2024-12-31'),
174 ('CON016', 'ACC006', 'MAT016', 3.50, '2024-12-31'),
175 ('CON017', 'ACC007', 'MAT017', 110.00, '2025-02-28'),
176 ('CON018', 'ACC008', 'MAT018', 18.00, '2025-03-31'),
177 ('CON019', 'ACC009', 'MAT019', 95.00, '2025-04-30'),
178 ('CON020', 'ACC010', 'MAT020', 14.00, '2024-09-30');
179
180 INSERT INTO Warehouse OVERRIDING SYSTEM VALUE VALUES
181 (1, 'MAT001', 1001, 'UT', 10000),
182 (2, 'MAT002', 1002, 'UT', 2000),
183 (3, 'MAT003', 1003, 'UT', 1500),
184 (4, 'MAT004', 1004, 'UT', 1000),
185 (5, 'MAT005', 1005, 'UT', 3000),
186 (6, 'MAT006', 1006, 'UT', 1000),

```

```

182 (7, 'MAT007', 1007, 'UT', 8000),
183 (8, 'MAT008', 1008, 'UT', 10000),
184 (9, 'MAT009', 1009, 'UT', 15000),
185 (10, 'MAT010', 1010, 'UT', 20000),
186 (11, 'MAT011', 1011, 'UT', 5000),
187 (12, 'MAT012', 1012, 'UT', 500),
188 (13, 'MAT013', 1013, 'UT', 500),
189 (14, 'MAT014', 1014, 'UT', 5000),
190 (15, 'MAT015', 1015, 'UT', 2000),
191 (16, 'MAT016', 1016, 'UT', 3000),
192 (17, 'MAT017', 1017, 'UT', 1500),
193 (18, 'MAT018', 1018, 'UT', 4000),
194 (19, 'MAT019', 1019, 'UT', 1000),
195 (20, 'MAT020', 1020, 'UT', 2500);
196
197 INSERT INTO RM_Transaction (Invoice_No, Item_ID, RM_Qty, Val)
    VALUES
198 (1001, 1, 10000, 150000.00),
199 (1002, 2, 2000, 120000.00),
200 (1003, 3, 1500, 180000.00),
201 (1004, 4, 1000, 50000.00),
202 (1005, 5, 3000, 75000.00),
203 (1006, 6, 1000, 40000.00),
204 (1007, 7, 8000, 95000.00),
205 (1008, 8, 10000, 110000.00),
206 (1009, 9, 15000, 200000.00),
207 (1010, 10, 20000, 130000.00),
208 (1011, 11, 5000, 60000.00),
209 (1012, 12, 500, 70000.00),
210 (1013, 13, 500, 80000.00),
211 (1014, 14, 5000, 90000.00),
212 (1015, 15, 2000, 100000.00),
213 (1016, 16, 3000, 110000.00),
214 (1017, 17, 1500, 120000.00),
215 (1018, 18, 4000, 130000.00),
216 (1019, 19, 1000, 140000.00),
217 (1020, 20, 2500, 150000.00);
218
219 INSERT INTO Material_Quality_Check (Report_ID, Item_ID,
    Analysis_Date, Analyst_Name, Sample_Size, Test, Limits,
    Results, Emp_ID) VALUES
220 ('MQC001', 1, '2023-02-02', 'Legacy_Name', 5, 'Potency', '>95%',
    'PASSED', 'EMP001'),
221 ('MQC002', 2, '2023-02-03', 'Legacy_Name', 2, 'Potency', '>98%',
    'PASSED', 'EMP002'),
222 ('MQC003', 3, '2023-02-04', 'Legacy_Name', 10, 'Purity', '>99%',
    'PASSED', 'EMP011'),
223 ('MQC004', 4, '2023-02-05', 'Legacy_Name', 10, 'Viscosity', '
    Standard', 'PASSED', 'EMP019'),
224 ('MQC005', 5, '2023-02-06', 'Legacy_Name', 5, 'Purity', '>99.9%',
    'FAILED', 'EMP001'),

```

```

225 ('MQC006', 6, '2023-02-07', 'Legacy_Name', 5, 'Potency', '>95%',
    , 'PASSED', 'EMP002'),
226 ('MQC007', 7, '2023-02-08', 'Legacy_Name', 20, 'Moisture', '<5%',
    , 'PASSED', 'EMP011'),
227 ('MQC008', 8, '2023-02-09', 'Legacy_Name', 20, 'Moisture', '<5%',
    , 'PASSED', 'EMP019'),
228 ('MQC009', 9, '2023-02-10', 'Legacy_Name', 30, 'Granularity', '
    Standard', 'PASSED', 'EMP001'),
229 ('MQC010', 10, '2023-02-11', 'Legacy_Name', 30, 'Granularity',
    'Standard', 'PASSED', 'EMP002'),
230 ('MQC011', 11, '2023-02-12', 'Legacy_Name', 5, 'Purity', '>99%',
    , 'PASSED', 'EMP011'),
231 ('MQC012', 12, '2023-02-13', 'Legacy_Name', 2, 'Assay', '
    95-105%', 'PASSED', 'EMP019'),
232 ('MQC013', 13, '2023-02-14', 'Legacy_Name', 2, 'Assay', '
    95-105%', 'PASSED', 'EMP001'),
233 ('MQC014', 14, '2023-02-15', 'Legacy_Name', 10, 'Moisture', '
    <2%', 'PASSED', 'EMP002'),
234 ('MQC015', 15, '2023-02-16', 'Legacy_Name', 5, 'Purity', '>99%',
    , 'PASSED', 'EMP011'),
235 ('MQC016', 16, '2023-02-17', 'Legacy_Name', 5, 'Purity', '>99%',
    , 'PASSED', 'EMP019'),
236 ('MQC017', 17, '2023-02-18', 'Legacy_Name', 5, 'Potency', '>98%',
    , 'PASSED', 'EMP001'),
237 ('MQC018', 18, '2023-02-19', 'Legacy_Name', 10, 'Potency', '
    >95%', 'PASSED', 'EMP002'),
238 ('MQC019', 19, '2023-02-20', 'Legacy_Name', 5, 'Potency', '>99%',
    , 'PASSED', 'EMP011'),
239 ('MQC020', 20, '2023-02-21', 'Legacy_Name', 10, 'Viscosity', '
    Standard', 'FAILED', 'EMP019');
240
241 INSERT INTO Formula_Master (Product_ID, Material_ID,
    Weight_per_tablet) VALUES
242 ('PRD001', 'MAT001', 500),
243 ('PRD001', 'MAT018', 200),
244 ('PRD002', 'MAT002', 200),
245 ('PRD003', 'MAT006', 1000),
246 ('PRD004', 'MAT004', 20),
247 ('PRD005', 'MAT005', 250),
248 ('PRD006', 'MAT012', 100),
249 ('PRD007', 'MAT003', 500),
250 ('PRD008', 'MAT006', 25),
251 ('PRD009', 'MAT017', 500),
252 ('PRD010', 'MAT018', 400),
253 ('PRD011', 'MAT019', 2),
254 ('PRD012', 'MAT001', 500),
255 ('PRD013', 'MAT001', 650),
256 ('PRD014', 'MAT013', 10),
257 ('PRD015', 'MAT002', 150),
258 ('PRD016', 'MAT005', 500),
259 ('PRD017', 'MAT004', 10),

```

```

260 ('PRD018', 'MAT003', 200),
261 ('PRD019', 'MAT019', 1),
262 ('PRD020', 'MAT017', 100);
263
264 INSERT INTO Batch (Batch_No, Batch_Size, Mfg_Date, Exp_Date,
265 Product_ID, Stock_Qty, UT_Q_A, Yield_Percentage) VALUES
266 (5001, 50000, '2023-03-01', '2025-02-28', 'PRD001', 49000, 'UT',
267 , 98.00),
268 (5002, 25000, '2023-03-02', '2025-02-28', 'PRD002', 24800, 'UT',
269 , 99.20),
270 (5003, 10000, '2023-03-03', '2024-03-02', 'PRD003', 9500, 'UT',
271 , 95.00),
272 (5004, 30000, '2023-03-04', '2025-03-03', 'PRD004', 29500, 'UT',
273 , 98.33),
274 (5005, 40000, '2023-03-05', '2024-03-04', 'PRD005', 39000, 'UT',
275 , 97.50),
276 (5006, 50000, '2023-03-06', '2026-03-05', 'PRD006', 48000, 'UT',
277 , 96.00),
278 (5007, 15000, '2023-03-07', '2024-03-06', 'PRD007', 14900, 'UT',
279 , 99.33),
280 (5008, 20000, '2023-03-08', '2024-03-07', 'PRD008', 19800, 'UT',
281 , 99.00),
282 (5009, 60000, '2023-03-09', '2025-03-08', 'PRD009', 59000, 'UT',
283 , 98.33),
284 (5010, 45000, '2023-03-10', '2026-03-09', 'PRD010', 44000, 'UT',
285 , 97.77),
286 (5011, 25000, '2023-03-11', '2025-03-10', 'PRD011', 24500, 'UT',
287 , 98.00),
288 (5012, 80000, '2023-03-12', '2025-03-11', 'PRD012', 79000, 'UT',
289 , 98.75),
290 (5013, 75000, '2023-03-13', '2025-03-12', 'PRD013', 74000, 'UT',
291 , 98.66),
292 (5014, 10000, '2023-03-14', '2025-03-13', 'PRD014', 9800, 'UT',
293 , 98.00),
294 (5015, 20000, '2023-03-15', '2025-03-14', 'PRD015', 19500, 'UT',
295 , 97.50),
296 (5016, 35000, '2023-03-16', '2025-03-15', 'PRD016', 34000, 'UT',
297 , 97.14),
298 (5017, 30000, '2023-03-17', '2025-03-16', 'PRD017', 29000, 'UT',
299 , 96.66),
300 (5018, 15000, '2023-03-18', '2024-03-17', 'PRD018', 14500, 'UT',
301 , 96.66),
302 (5019, 25000, '2023-03-19', '2025-03-18', 'PRD019', 24000, 'UT',
303 , 96.00),
304 (5020, 10000, '2023-03-20', '2024-03-19', 'PRD020', 9500, 'UT',
305 , 95.00);
306
307 INSERT INTO Material_Dispensing (Batch_No, Item_ID,
308 Quantity_Issued) VALUES
309 (5001, 7, 500), -- Dispensing Sodium Starch Glycolate (
310 Disintegrant)

```

```

288 (5001, 8, 200), -- Dispensing Methyl Cellulose (Binder)
289 (5001, 15, 50), -- Dispensing Magnesium Stearate (Lubricant)
290 (5001, 1, 2500),
291 (5002, 2, 500),
292 (5003, 6, 1000),
293 (5004, 4, 600),
294 (5005, 5, 1000),
295 (5006, 12, 50),
296 (5007, 3, 750),
297 (5008, 6, 500),
298 (5009, 17, 3000),
299 (5010, 18, 1800),
300 (5011, 19, 50),
301 (5012, 1, 4000),
302 (5013, 1, 4875),
303 (5014, 13, 10),
304 (5015, 2, 300),
305 (5016, 5, 1750),
306 (5017, 4, 300),
307 (5018, 3, 300),
308 (5019, 19, 25),
309 (5020, 17, 100);
310
311 INSERT INTO Production_Log (Batch_No, Equipment_ID, Emp_ID,
    Process_Stage, Start_Time, End_Time) VALUES
312 (5001, 'EQ001', 'EMP003', 'Dry_Mixing', '2023-03-01_08:00:00',
    '2023-03-01_10:30:00'),
313 (5002, 'EQ002', 'EMP004', 'Wet_Granulation', '2023-03-02_
    11:00:00', '2023-03-02_15:00:00'),
314 (5003, 'EQ009', 'EMP005', 'Liquid_Processing', '2023-03-03_
    08:00:00', '2023-03-03_16:30:00'),
315 (5004, 'EQ004', 'EMP003', 'Compression', '2023-03-04_09:00:00',
    '2023-03-04_17:00:00'),
316 (5005, 'EQ009', 'EMP004', 'Liquid_Processing', '2023-03-05_
    08:00:00', '2023-03-05_11:00:00'),
317 (5006, 'EQ004', 'EMP017', 'Compression', '2023-03-06_08:00:00',
    '2023-03-06_14:00:00'),
318 (5007, 'EQ005', 'EMP003', 'Compression', '2023-03-07_10:00:00',
    '2023-03-07_18:00:00'),
319 (5008, 'EQ011', 'EMP004', 'Capsule_Filling', '2023-03-08_
    08:00:00', '2023-03-08_16:00:00'),
320 (5009, 'EQ005', 'EMP017', 'Compression', '2023-03-09_09:00:00',
    '2023-03-09_17:00:00'),
321 (5010, 'EQ004', 'EMP003', 'Compression', '2023-03-10_08:00:00',
    '2023-03-10_15:00:00'),
322 (5011, 'EQ006', 'EMP004', 'Coating', '2023-03-11_11:00:00', '
    2023-03-11_19:00:00'),
323 (5012, 'EQ005', 'EMP017', 'Compression', '2023-03-12_08:00:00',
    '2023-03-12_20:00:00'),
324 (5013, 'EQ004', 'EMP003', 'Compression', '2023-03-13_08:00:00',
    '2023-03-13_19:00:00'),

```



```

325 (5014, 'EQ011', 'EMP004', 'Capsule_Filling', '2023-03-14_
      09:00:00', '2023-03-14_13:00:00'),
326 (5015, 'EQ011', 'EMP017', 'Capsule_Filling', '2023-03-15_
      14:00:00', '2023-03-15_18:00:00'),
327 (5016, 'EQ011', 'EMP003', 'Capsule_Filling', '2023-03-16_
      08:00:00', '2023-03-16_16:00:00'),
328 (5017, 'EQ004', 'EMP004', 'Compression', '2023-03-17_10:00:00',
      '2023-03-17_16:00:00'),
329 (5018, 'EQ009', 'EMP017', 'Liquid_Processing', '2023-03-18_
      08:00:00', '2023-03-18_12:00:00'),
330 (5019, 'EQ005', 'EMP003', 'Compression', '2023-03-19_13:00:00',
      '2023-03-19_18:00:00'),
331 (5020, 'EQ009', 'EMP004', 'Liquid_Processing', '2023-03-20_
      08:00:00', '2023-03-20_11:00:00');
332
333 INSERT INTO Product_Quality_Check (Report_ID, Batch_No,
      Analysis_Date, Analyst_Name, Sample_Size, Process_State,
      Test, Limits, Results, Emp_ID) VALUES
334 ('PQC001', 5001, '2023-03-02', 'Legacy_Name', 100, 'Finished_
      Good', 'Dissolution', '>85%_in_30m', 'PASSED', 'EMP001'),
335 ('PQC002', 5002, '2023-03-03', 'Legacy_Name', 50, 'Finished_
      Good', 'Assay', '95%-105%', 'PASSED', 'EMP002'),
336 ('PQC003', 5003, '2023-03-04', 'Legacy_Name', 20, 'Finished_
      Good', 'Viscosity', 'Standard', 'FAILED', 'EMP011'),
337 ('PQC004', 5004, '2023-03-05', 'Legacy_Name', 100, 'Finished_
      Good', 'Hardness', '4-8_kp', 'PASSED', 'EMP019'),
338 ('PQC005', 5005, '2023-03-06', 'Legacy_Name', 20, 'Finished_
      Good', 'pH_Level', '5.0-7.0', 'PASSED', 'EMP001'),
339 ('PQC006', 5006, '2023-03-07', 'Legacy_Name', 100, 'Finished_
      Good', 'Assay', '90%-110%', 'PASSED', 'EMP002'),
340 ('PQC007', 5007, '2023-03-08', 'Legacy_Name', 100, 'Finished_
      Good', 'Dissolution', '>80%_in_30m', 'PASSED', 'EMP011'),
341 ('PQC008', 5008, '2023-03-09', 'Legacy_Name', 50, 'Finished_
      Good', 'Disintegration', '<15m', 'PASSED', 'EMP019'),
342 ('PQC009', 5009, '2023-03-10', 'Legacy_Name', 100, 'Finished_
      Good', 'Assay', '95%-105%', 'PASSED', 'EMP001'),
343 ('PQC010', 5010, '2023-03-11', 'Legacy_Name', 100, 'Finished_
      Good', 'Hardness', '4-8_kp', 'PASSED', 'EMP002'),
344 ('PQC011', 5011, '2023-03-12', 'Legacy_Name', 50, 'Finished_
      Good', 'Assay', '90%-110%', 'FAILED', 'EMP011'),
345 ('PQC012', 5012, '2023-03-13', 'Legacy_Name', 100, 'Finished_
      Good', 'Dissolution', '>85%_in_30m', 'PASSED', 'EMP019'),
346 ('PQC013', 5013, '2023-03-14', 'Legacy_Name', 100, 'Finished_
      Good', 'Hardness', '4-8_kp', 'PASSED', 'EMP001'),
347 ('PQC014', 5014, '2023-03-15', 'Legacy_Name', 50, 'Finished_
      Good', 'Disintegration', '<15m', 'PASSED', 'EMP002'),
348 ('PQC015', 5015, '2023-03-16', 'Legacy_Name', 50, 'Finished_
      Good', 'Assay', '95%-105%', 'PASSED', 'EMP011'),
349 ('PQC016', 5016, '2023-03-17', 'Legacy_Name', 100, 'Finished_
      Good', 'Disintegration', '<15m', 'PASSED', 'EMP019'),
350 ('PQC017', 5017, '2023-03-18', 'Legacy_Name', 100, 'Finished_

```



```

351      Good', 'Hardness', '4-8_kp', 'PASSED', 'EMP001'),
      ('PQC018', 5018, '2023-03-19', 'Legacy_Name', 20, 'Finished_
352      Good', 'pH_Level', '5.0-7.0', 'PASSED', 'EMP002'),
      ('PQC019', 5019, '2023-03-20', 'Legacy_Name', 100, 'Finished_
353      Good', 'Assay', '95%-105%', 'FAILED', 'EMP011'),
      ('PQC020', 5020, '2023-03-21', 'Legacy_Name', 20, 'Finished_
      Good', 'Viscosity', 'Standard', 'PASSED', 'EMP019');
354
355 INSERT INTO FG_Transaction (Invoice_No, Batch_No, Sale_Qty, Val
      ) VALUES
356 (2001, 5001, 20000, 250000.00),
357 (2002, 5002, 10000, 300000.00),
358 (2003, 5003, 5000, 150000.00),
359 (2004, 5004, 15000, 400000.00),
360 (2005, 5005, 20000, 500000.00),
361 (2006, 5006, 25000, 200000.00),
362 (2007, 5007, 10000, 350000.00),
363 (2008, 5008, 10000, 450000.00),
364 (2009, 5009, 30000, 120000.00),
365 (2010, 5010, 20000, 280000.00),
366 (2011, 5011, 12000, 110000.00),
367 (2012, 5012, 40000, 220000.00),
368 (2013, 5013, 35000, 330000.00),
369 (2014, 5014, 5000, 440000.00),
370 (2015, 5015, 10000, 550000.00),
371 (2016, 5016, 15000, 660000.00),
372 (2017, 5017, 15000, 770000.00),
373 (2018, 5018, 7000, 880000.00),
374 (2019, 5019, 12000, 990000.00),
375 (2020, 5020, 4000, 1000000.00);
376
377 INSERT INTO Product_Recall (Recall_ID, Batch_No, Date_Initiated
      , Reason, Qty_Recalled) VALUES
378 ('REC001', 5003, '2023-05-01', 'Failed_viscosity_stability_
      tests.', 5000),
379 ('REC002', 5011, '2023-05-02', 'Assay_degraded_below_90%_in_
      market.', 12000),
380 ('REC003', 5019, '2023-05-03', 'Contamination_warning_from_raw_
      supplier.', 12000),
381 ('REC004', 5001, '2023-05-04', 'Packaging_label_misprint.',
      1000),
382 ('REC005', 5002, '2023-05-05', 'Seal_integrity_failure_reported
      .', 500),
383 ('REC006', 5004, '2023-05-06', 'Customer_reports_of_bad_odor.',
      2000),
384 ('REC007', 5005, '2023-05-07', 'pH_level_dropped_out_of_spec.',
      3000),
385 ('REC008', 5006, '2023-05-08', 'Voluntary_recall_-_updated_
      formulation.', 4000),
386 ('REC009', 5007, '2023-05-09', 'Failed_3-month_dissolution_test
      .', 1500),

```

```

387 ('REC010', 5008, '2023-05-10', 'Capsule_shells_brittle.', 2500)
388 ('REC011', 5009, '2023-05-11', 'Incorrect_expiry_date_printed.',
389 , 5000),
390 ('REC012', 5010, '2023-05-12', 'Discoloration_of_tablets.',
391 , 1000),
392 ('REC013', 5012, '2023-05-13', 'Failed_friability_test_post-
393 shipping.', 8000),
394 ('REC014', 5013, '2023-05-14', 'Blister_pack_foil_delamination.
395 ', 4500),
396 ('REC015', 5014, '2023-05-15', 'Sub-potent_assay_reported_by_
397 distributor.', 500),
398 ('REC016', 5015, '2023-05-16', 'Cross-contamination_suspected.',
399 , 2000),
400 ('REC017', 5016, '2023-05-17', 'Failed_microbial_limits_test.',
401 , 3500),
402 ('REC018', 5017, '2023-05-18', 'Foreign_particle_found_in_
403 bottle.', 100),
404 ('REC019', 5018, '2023-05-19', 'Syrup_crystallized_prematurely.
405 ', 3000),
406 ('REC020', 5020, '2023-05-20', 'Dropper_malfunction_reported.',
407 , 1500);
408
409 INSERT INTO Maintenance_Log (Maintenance_ID, Equipment_ID,
410 Emp_ID, Maintenance_Date, Cost) VALUES
411 ('ML001', 'EQ003', 'EMP006', '2023-01-11', 4500.00),
412 ('ML002', 'EQ015', 'EMP007', '2023-01-31', 1200.50),
413 ('ML003', 'EQ001', 'EMP015', '2023-02-15', 850.00),
414 ('ML004', 'EQ002', 'EMP006', '2023-02-20', 300.00),
415 ('ML005', 'EQ004', 'EMP007', '2023-03-05', 1500.00),
416 ('ML006', 'EQ005', 'EMP015', '2023-03-10', 900.00),
417 ('ML007', 'EQ006', 'EMP006', '2023-03-15', 2100.00),
418 ('ML008', 'EQ007', 'EMP007', '2023-03-20', 400.00),
419 ('ML009', 'EQ008', 'EMP015', '2023-03-25', 650.00),
420 ('ML010', 'EQ009', 'EMP006', '2023-04-01', 5000.00),
421 ('ML011', 'EQ010', 'EMP007', '2023-04-05', 750.00),
422 ('ML012', 'EQ011', 'EMP015', '2023-04-10', 1100.00),
423 ('ML013', 'EQ012', 'EMP006', '2023-04-15', 300.00),
424 ('ML014', 'EQ013', 'EMP007', '2023-04-20', 4200.00),
425 ('ML015', 'EQ014', 'EMP015', '2023-04-25', 800.00),
426 ('ML016', 'EQ016', 'EMP006', '2023-05-01', 6000.00),
427 ('ML017', 'EQ017', 'EMP007', '2023-05-05', 2500.00),
428 ('ML018', 'EQ018', 'EMP015', '2023-05-10', 1800.00),
429 ('ML019', 'EQ019', 'EMP006', '2023-05-15', 950.00),
430 ('ML020', 'EQ020', 'EMP007', '2023-05-20', 450.00);

```

8 Relevant SQL Queries & Enterprise Analytics

This section showcases the analytical capabilities of **PharmaChain-DB**. These queries transform complex, highly structured data into meaningful insights for **business decisions, compliance tracking, and supply chain monitoring**.

1. Revenue Analytics: "Which products are currently making us the most money?"

Relational Algebra:

$$\gamma_{\text{Product_Name, Product_Type, COUNT(Batch_No), SUM(Sale_Qty), SUM(Val)}}(\text{FG_Transaction} \bowtie \text{Batch} \bowtie \text{Product_Master})$$

```
1 SELECT
2     pm.Product_Name ,
3     pm.Product_Type ,
4     COUNT(DISTINCT fgt.Batch_No) AS Batches_Sold ,
5     SUM(fgt.Sale_Qty) AS Total_Units_Sold ,
6     SUM(fgt.Val) AS Total_Revenue_INR
7 FROM FG_Transaction fgt
8 JOIN Batch b ON fgt.Batch_No = b.Batch_No
9 JOIN Product_Master pm ON b.Product_ID = pm.Product_ID
10 GROUP BY pm.Product_Name , pm.Product_Type
11 ORDER BY Total_Revenue_INR DESC;
```

Business Significance:

- Identifies high-margin pharmaceutical products driving the company's primary revenue.
- Guides future production planning by highlighting the most demanded medicines.
- Evaluates financial performance across different physical product types (e.g., Tablets vs. Syrups).

Query Output:

Data Output Messages Notifications					
	product_name character varying (20)	product_type character varying (20)	batches_sold bigint	total_units_sold numeric	total_revenue_inr numeric
1	Cipro-Drops	Liquid	1	4000	1000000.00
2	Glime-M	Tablet	1	12000	990000.00
3	Zithro-Susp	Syrup	1	7000	880000.00
4	Atorva-10	Tablet	1	15000	770000.00
5	Amox-500	Capsule	1	15000	660000.00
6	Ome-Protect	Capsule	1	10000	550000.00
7	Amoxyn	Syrup	1	20000	500000.00
8	Aller-D	Capsule	1	10000	450000.00
9	Vit-D Core	Capsule	1	5000	440000.00
10	Atorvastatin 20	Tablet	1	15000	400000.00
11	Zithrol 500	Tablet	1	10000	350000.00
12	Para-650	Tablet	1	35000	330000.00
13	Omsergy	Capsule	1	10000	300000.00
14	Parabufen	Tablet	1	21000	300000.00
15	Ibupro-G	Tablet	1	20000	280000.00
16	Para-500	Tablet	1	40000	220000.00
17	Vitarich	Tablet	1	25000	200000.00
18	Exicof	Syrup	1	5000	150000.00
19	Cipro-Max	Tablet	1	30000	120000.00
20	Glimeta 2	Tablet	1	12000	110000.00

Figure 1: Output: Revenue Analytics Dashboard

2. The QA Audit: "Show me a timeline of failed batches and who evaluated them."

Relational Algebra:

$$\tau_{\text{Analysis_Date}} \downarrow \left(\pi_{\text{Date, Batch_No, Name, Test, Limits, Results, Emp_Name}} \left(\sigma_{\text{Results}='FAILED'} (\text{PQC} \bowtie \text{Batch} \bowtie \text{PM} \bowtie \text{EM}) \right) \right)$$

```

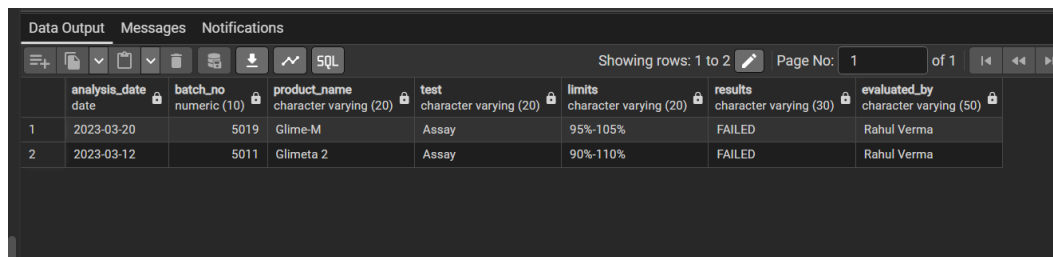
1 SELECT
2     pqc.Analysis_Date ,
3     pqc.Batch_No ,
4     pm.Product_Name ,
5     pqc.Test ,
6     pqc.Limits ,
7     pqc.Results ,
8     em.Emp_Name AS Evaluated_By
9 FROM Product_Quality_Check pqc
10 JOIN Batch b ON pqc.Batch_No = b.Batch_No
11 JOIN Product_Master pm ON b.Product_ID = pm.Product_ID
12 JOIN Employee_Master em ON pqc.Emp_ID = em.Emp_ID
13 WHERE pqc.Results = 'FAILED'
14 ORDER BY pqc.Analysis_Date DESC;

```

Business Significance:

- Traces systemic quality control failures back to specific production runs.
- Ensures total accountability by linking a specific lab analyst to a failed result.
- Provides critical documentation for FDA compliance and internal quality reviews.

Query Output:



The screenshot shows a data output window with a table of quality assurance failure timeline data. The table has 8 columns: analysis_date, batch_no, product_name, test, limits, results, and evaluated_by. The data is displayed in two rows, both showing 'FAILED' results.

	analysis_date date	batch_no numeric (10)	product_name character varying (20)	test character varying (20)	limits character varying (20)	results character varying (30)	evaluated_by character varying (50)
1	2023-03-20	5019	Glime-M	Assay	95%-105%	FAILED	Rahul Verma
2	2023-03-12	5011	Glimeta 2	Assay	90%-110%	FAILED	Rahul Verma

Figure 2: Output: Quality Assurance Failure Timeline

3. The Supply Chain Check: "Who are our suppliers and what materials are they under contract to give us?"

Relational Algebra:

$$\pi_{\text{Account_Name, Phone_No, Material_Name, Agreed_Price, Valid_Until}}(\sigma_{\text{Valid_Until} \geq \text{CURRENT_DATE}}(\text{Contract} \bowtie \text{Account} \bowtie \text{Material}))$$

```
1 SELECT
2     am.Account_Name AS Supplier_Name ,
3     am.Phone_No ,
4     mm.Material_Name ,
5     sc.Agreed_Price ,
6     sc.Valid_Until
7 FROM Supplier_Contract sc
8 JOIN Account_Master am ON sc.Account_No = am.Account_No
9 JOIN Material_Master mm ON sc.Material_ID = mm.Material_ID
10 WHERE sc.Valid_Until >= CURRENT_DATE
11 ORDER BY am.Account_Name;
```

Business Significance:

- Ensures active supply lines by filtering out expired vendor contracts.
- Monitors negotiated procurement pricing to prevent accidental over-billing.
- Provides procurement officers with an instant directory of active chemical vendors.

Query Output:

Data Output Messages Notifications					
Showing rows: 1 to 2 Page					
	supplier_name character varying (50)	phone_no character varying (13)	material_name character varying (30)	agreed_price numeric (10,2)	valid_until date
1	EuroPharma Raw	442071234567	Sucrose	2.00	2026-12-31
2	Zhengzhou Organics	861012345678	Starch	1.50	2026-12-31

Figure 3: Output: Active Supplier Contracts

4. The Maintenance Alarm: "Which machines cost us the most money in repairs?"

Relational Algebra:

$$\gamma_{\text{Equipment_ID, Name, Type, COUNT(Maintenance_ID), SUM(Cost)}}(\text{Maintenance_Log} \bowtie \text{Equipment_Master})$$

```
1 SELECT
2     eq.Equipment_ID ,
3     eq.Equipment_Name ,
4     eq.Equipment_Type ,
5     COUNT(ml.Maintenance_ID) AS Times_Repaired ,
6     SUM(ml.Cost) AS Total_Repair_Cost
7 FROM Maintenance_Log ml
8 JOIN Equipment_Master eq ON ml.Equipment_ID = eq.Equipment_ID
9 GROUP BY eq.Equipment_ID , eq.Equipment_Name , eq.Equipment_Type
10 ORDER BY Total_Repair_Cost DESC;
```

Business Significance:

- Identifies rapidly depreciating assets on the factory floor.
- Calculates the Total Cost of Ownership (TCO) for heavy machinery.
- Guides capital expenditure decisions (e.g., deciding whether to replace a machine vs. continually repairing it).

Query Output:

Data Output Messages Notifications						
Showing rows: 1 to 20 Page						
	equipment_Id [PK] character varying (20)	equipment_name character varying (50)	equipment_type character varying (30)	times_repaired bigint	total_repair_cost numeric	
1	EQ016	Purified Water System	Utility	1	6000.00	
2	EQ009	Liquid Mixing Tank 50...	Liquid Processor	1	5000.00	
3	EQ003	Fluid Bed Granulator 1	Granulator	1	4500.00	
4	EQ013	Fluid Bed Granulator 2	Granulator	1	4200.00	
5	EQ017	Air Handling Unit	Utility	1	2500.00	
6	EQ006	Auto Film Coater	Coater	1	2100.00	
7	EQ018	Industrial Autoclave	Sterilizer	1	1800.00	
8	EQ004	Rotary Tablet Press 1	Compressor	1	1500.00	
9	EQ015	Cartoning Machine	Packaging	1	1200.50	
10	EQ011	Capsule Filling Machine	Filler	1	1100.00	
11	EQ019	Metal Detector Line	QA Equipment	1	950.00	
12	EQ005	Rotary Tablet Press 2	Compressor	1	900.00	
13	EQ001	High Shear Mixer A	Mixer	1	850.00	
14	EQ014	Tablet Checking Mach...	QA Equipment	1	800.00	
15	EQ010	Syrup Bottling Station	Packaging	1	750.00	
16	EQ008	Bottle Filling Line B	Packaging	1	650.00	
17	EQ020	Shrink Wrapping Mac...	Packaging	1	450.00	
18	EQ007	Blister Packaging Line...	Packaging	1	400.00	
19	EQ012	V-Cone Blender	Mixer	1	300.00	
20	EQ002	High Shear Mixer B	Mixer	1	300.00	

Figure 4: Output: Equipment Repair Cost Analysis

5. The Efficiency Trap: "Which production stages take the longest time to finish?"

Relational Algebra:

$$\gamma_{\text{Process_Stage}, \text{COUNT}(\text{Log_ID}), \text{AVG}(\text{End_Time} - \text{Start_Time})}(\text{Production_Log})$$

```

1 SELECT
2     Process_Stage ,
3     COUNT(Log_ID) AS Total_Runs_Logged ,
4     AVG(End_Time - Start_Time) AS Average_Processing_Duration
5 FROM Production_Log
6 GROUP BY Process_Stage
7 ORDER BY Average_Processing_Duration DESC;

```

Business Significance:

- Highlights operational bottlenecks within the manufacturing pipeline.
- Assists plant managers in optimizing employee shift scheduling around long processes.
- Sets baseline performance metrics for future factory automation upgrades.

Query Output:

Data Output Messages Notifications			
	process_stage character varying (30) 🔒	total_runs_logged bigint 🔒	average_processing_duration interval 🔒
1	Coating	1	08:00:00
2	Compression	9	07:53:20
3	Capsule Filling	4	06:00:00
4	Liquid Processing	4	04:37:30
5	Wet Granulation	1	04:00:00
6	Dry Mixing	1	02:30:00

Figure 5: Output: Manufacturing Stage Durations

6. The Compliance Drill: "Show me all active audit trail evidence logs."

Relational Algebra:

$$\tau_{\text{Change_Date}} \downarrow \left(\pi_{\text{Date, Report_ID, Action_Type, Old_Result, New_Result, User}} \right. \\ \left. (\text{QC_Audit_Log}) \right)$$

```

1 SELECT
2     Change_Date ,
3     Report_ID ,
4     Action_Type ,
5     Old_Result AS Before_Change ,
6     New_Result AS After_Change ,
7     Changed_By AS Database_User
8 FROM QC_Audit_Log
9 ORDER BY Change_Date DESC;
```

Business Significance:

- Satisfies the strict regulatory requirements of FDA 21 CFR Part 11.
- Acts as a tamper-evident security layer against unauthorized data manipulation.
- Provides irrefutable digital evidence during surprise government safety inspections.

Query Output:

Data Output Messages Notifications						
	change_date timestamp without time zone	report_id character varying (20)	action_type character varying (10)	before_change character varying (50)	after_change character varying (50)	database_user character varying (50)
1	2026-05-16 16:59:24.102317	PQC004	UPDATE	PASSED	RECALLED	postgres
2	2026-05-16 16:11:09.077744	PQC005	DELETE	PASSED	RECORD DESTROYED	postgres
3	2026-05-16 16:10:16.005322	PQC003	UPDATE	FAILED	PASSED	postgres

Figure 6: Output: Quality Control Shadow Ledger

7. The Financial Leak Tracker: "What are our total financial losses from recalled batches?"

Relational Algebra:

$$\tau_{\text{Date_Initiated}} \downarrow \left(\pi_{\text{Recall_ID}, \text{Date}, \text{Batch_No}, \text{Name}, \text{Qty_Recalled}, \text{Reason}} \right. \\ \left. (\text{Product_Recall} \bowtie \text{Batch} \bowtie \text{Product_Master}) \right)$$

```

1 SELECT
2     pr.Recall_ID ,
3     pr.Date_Initiated ,
4     pr.Batch_No ,
5     pm.Product_Name ,
6     pr.Qty_Recalled AS Units_Destroyed ,
7     pr.Reason
8 FROM Product_Recall pr
9 JOIN Batch b ON pr.Batch_No = b.Batch_No
10 JOIN Product_Master pm ON b.Product_ID = pm.Product_ID
11 ORDER BY pr.Date_Initiated DESC;
```

Business Significance:

- Quantifies enterprise risk and financial exposure resulting from catastrophic medicine failures.
- Tracks recurring safety issues to determine if specific formulas need to be entirely discontinued.
- Provides absolute transparency for executive stakeholders and corporate reporting.

Query Output:

Data Output Messages Notifications						
Showing rows: 1 to 21 Page No: 1 of 1						
	recall_id character varying (20)	date_initiated date	batch_no numeric (10)	product_name character varying (20)	units_destroyed numeric (10)	reason character varying (255)
1	REC-5004-A	2026-05-16	5004	Atorvastatin 20	29500	CRITICAL: Stability test failed at 6 months. Active ingredient degra...
2	REC020	2023-05-20	5020	Cipro-Drops	1500	Dropper malfunction reported.
3	REC019	2023-05-19	5018	Zithro-Susp	3000	Syrup crystallized prematurely.
4	REC018	2023-05-18	5017	Atorva-10	100	Foreign particle found in bottle.
5	REC017	2023-05-17	5016	Amox-500	3500	Failed microbial limits test.
6	REC016	2023-05-16	5015	Ome-Protect	2000	Cross-contamination suspected.
7	REC015	2023-05-15	5014	Vit-D Core	500	Sub-potent assay reported by distributor.
8	REC014	2023-05-14	5013	Para-650	4500	Blister pack foil delamination.
9	REC013	2023-05-13	5012	Para-500	8000	Failed friability test post-shipping.
10	REC012	2023-05-12	5010	Ibupro-G	1000	Discoloration of tablets.
11	REC011	2023-05-11	5009	Cipro-Max	5000	Incorrect expiry date printed.
12	REC010	2023-05-10	5008	Aller-D	2500	Capsule shells brittle.
13	REC009	2023-05-09	5007	Zithrol 500	1500	Failed 3-month dissolution test.
14	REC008	2023-05-08	5006	Vitarich	4000	Voluntary recall - updated formulation.
15	REC007	2023-05-07	5005	Amoxyn	3000	pH level dropped out of spec.
16	REC006	2023-05-06	5004	Atorvastatin 20	2000	Customer reports of bad odor.
17	REC005	2023-05-05	5002	Omsergy	500	Seal integrity failure reported.
18	REC004	2023-05-04	5001	Parabufen	1000	Packaging label misprint.
19	REC003	2023-05-03	5019	Glime-M	12000	Contamination warning from raw supplier.
20	REC002	2023-05-02	5011	Glimeta 2	12000	Assay degraded below 90% in market.
21	REC001	2023-05-01	5003	Exicof	5000	Failed viscosity stability tests.

Figure 7: Output: Recalled Products Registry

8. The Warehouse Audit: "What is our current storage profile for hazardous chemicals?"

Relational Algebra:

$$\tau_{\text{Stock}} \downarrow \left(\pi_{\text{Item_ID, Name, Stock, UOM, Storage, isHazardous, isInflammable}} \left(\sigma_{\text{isHazardous=T} \vee \text{isInflammable=T}} (\text{Warehouse} \bowtie \text{Material_Master}) \right) \right)$$

```

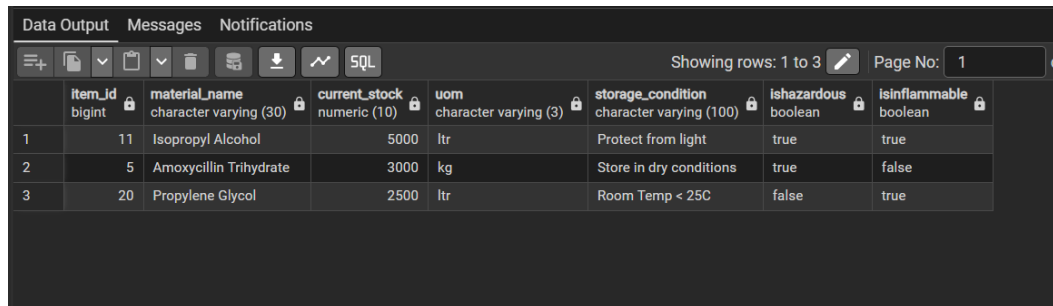
1 SELECT
2     w.Item_ID,
3     mm.Material_Name,
4     w.Stock AS Current_Stock,
5     mm.UOM,
6     mm.Storage_Condition,
7     mm.isHazardous,
8     mm.isInflammable
9 FROM Warehouse w
10 JOIN Material_Master mm ON w.Material_ID = mm.Material_ID
11 WHERE mm.isHazardous = TRUE OR mm.isInflammable = TRUE
12 ORDER BY w.Stock DESC;
```

Business Significance:

- Ensures total compliance with OSHA and environmental safety storage regulations.
- Prevents the dangerous physical stockpiling of highly volatile chemicals in a single zone.

→ Instantly alerts warehouse managers to high-risk storage sectors requiring specialized climate control.

Query Output:



The screenshot shows a database query output interface with a dark theme. At the top, there are tabs for 'Data Output', 'Messages', and 'Notifications'. Below the tabs is a toolbar with icons for various actions like expand, copy, download, and a 'SQL' button. To the right of the toolbar, it says 'Showing rows: 1 to 3' and 'Page No: 1'. The main content is a table with 8 columns: 'item_id', 'material_name', 'current_stock', 'uom', 'storage_condition', 'ishazardous', and 'isflammable'. The table contains 3 rows of data.

	item_id bigint	material_name character varying (30)	current_stock numeric (10)	uom character varying (3)	storage_condition character varying (100)	ishazardous boolean	isflammable boolean
1	11	Isopropyl Alcohol	5000	ltr	Protect from light	true	true
2	5	Amoxicillin Trihydrate	3000	kg	Store in dry conditions	true	false
3	20	Propylene Glycol	2500	ltr	Room Temp < 25C	false	true

Figure 8: Output: Hazardous Material Inventory

9 Active Database Automation: Triggers

9.1 1. The Warehouse Automator (trg_deduct_stock_on_dispense)

Execution Phase: AFTER INSERT on Material_Dispensing

Why is this important?

- **Zero Manual Math:** The moment a factory worker logs that they have taken raw materials for a medicine batch, this trigger automatically does the subtraction in the background. It instantly updates the physical warehouse stock without needing a human to remember to do it.
- **The Negative Stock Fail-Safe:** It acts as a hard security guard. If someone accidentally types an extra zero and tries to take out more stock than what actually exists on the shelf, the trigger catches it and **blocks the transaction entirely**. This guarantees the database never shows impossible "negative" inventory.

```
1  --
2  -- FUNCTION: Automate Warehouse Stock Deduction
3  -- DESCRIPTION: When a batch takes material, deduct it from the
4  --                warehouse.
5  --                If there isn't enough stock, block the
6  --                transaction.
7  --
8  CREATE OR REPLACE FUNCTION automate_warehouse_stock()
9  RETURNS TRIGGER
10 LANGUAGE plpgsql
11 AS $$
12 DECLARE
13     v_current_stock NUMERIC(10);
14 BEGIN
15     -- 1. Find out how much stock we currently have for this
16     --    specific item
17     SELECT Stock INTO v_current_stock
18     FROM Warehouse
19     WHERE Item_ID = NEW.Item_ID;
20
21     -- 2. Safety Check: Do we have enough stock to fulfill this
22     --    request?
23     IF v_current_stock < NEW.Quantity_Issued THEN
24         RAISE EXCEPTION 'Transaction Blocked: Not enough stock!
25         Item % only has % units left, but you tried to
26         dispense %.',
27             NEW.Item_ID, v_current_stock, NEW.
28             Quantity_Issued;
29     END IF;
30
31     -- 3. If we have enough stock, safely deduct it from the
32     --    Warehouse
33     UPDATE Warehouse
```

```

26     SET Stock = Stock - NEW.Quantity_Issued
27     WHERE Item_ID = NEW.Item_ID;
28
29     -- 4. Allow the original INSERT into Material_Dispensing to
30     finish
31     RETURN NEW;
32 END;
33 $$;
34
35 CREATE TRIGGER trg_deduct_stock_on_dispense
36 AFTER INSERT ON Material_Dispensing
37 FOR EACH ROW
EXECUTE FUNCTION automate_warehouse_stock();

```

Live Test Case (Triggering the Fail-Safe):

```

1  -- Attempting to dispense an impossible amount of material
2  (999,999 units)
3  INSERT INTO Material_Dispensing (Batch_No, Item_ID,
   Quantity_Issued)
VALUES (5001, 2, 999999);

```

Test Output:

Data Output	Messages	Notifications
ERROR: Transaction Blocked: Not enough stock! Item 2 only has 2000 units left, but you tried to dispense 999999. CONTEXT: PL/pgSQL function automate_warehouse_stock() line 12 at RAISE SQL state: P0001		

Figure 9: Output: Trigger blocking the negative stock transaction

9.2 2. The FDA Compliance Blocker (trg_prevent_bad_sales)

Execution Phase: BEFORE INSERT on FG_Transaction

Why is this important?

- **The Ultimate Safety Net:** This trigger acts as the final gatekeeper before medicine leaves the factory. Whenever a hospital or distributor attempts to purchase a batch, the database automatically cross-references the official Quality Control lab results in real-time.
- **Absolute Enforcement:** If a batch failed its lab test, or if it simply hasn't been tested yet, the trigger throws a hard exception. It makes it technologically impossible to sell toxic or unverified medicine, ensuring strict FDA compliance regardless of human error or frontend software bugs.

```
1  --
2  -- FUNCTION: FDA Compliance Enforcer
3  -- DESCRIPTION: Prevents the sale of any batch in
4  --                FG_Transaction if it
5  --                failed Quality Control or hasn't been tested
6  --                yet.
7  --
8  --
9  --
10 --
11 --
12 --
13 --
14 --
15 --
16 --
17 --
18 --
19 --
20 --
21 --
22 --
23 --
24 --
25 --
26 --
27 --
28 --
29 --

CREATE OR REPLACE FUNCTION enforce_quality_control()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_qa_result VARCHAR(30);
BEGIN
    -- 1. Look up the official lab result for the batch they
    --    are trying to sell
    SELECT Results INTO v_qa_result
    FROM Product_Quality_Check
    WHERE Batch_No = NEW.Batch_No;

    -- 2. Safety Rule A: Has it been tested at all?
    IF v_qa_result IS NULL THEN
        RAISE EXCEPTION 'COMPLIANCE_BLOCK: Batch % has not been
            tested by the QC Lab yet. Sale is illegal!', NEW.
            Batch_No;
    END IF;

    -- 3. Safety Rule B: Did it fail the test?
    IF v_qa_result = 'FAILED' THEN
        RAISE EXCEPTION 'SAFETY_BLOCK: Batch % FAILED quality
            control. This batch must be quarantined, not sold!',
            NEW.Batch_No;
    END IF;

    -- 4. If it exists and didn't fail (i.e., 'PASSED'), allow
    --    the sale to proceed
```

```

30     RETURN NEW;
31 END;
32 $$;
33
34 -- Bind the trigger to fire BEFORE the sale is finalized
35 CREATE TRIGGER trg_prevent_bad_sales
36 BEFORE INSERT ON FG_Transaction
37 FOR EACH ROW
38 EXECUTE FUNCTION enforce_quality_control();

```

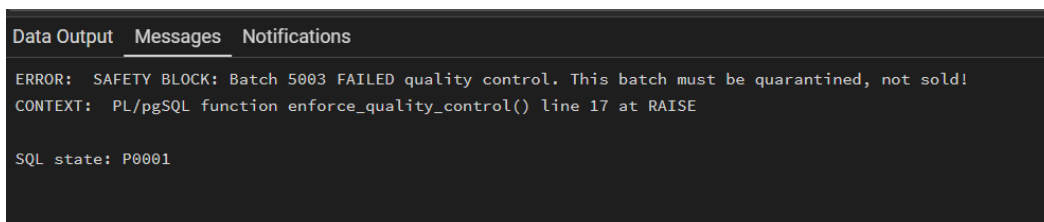
Live Test Case (Triggering the Compliance Block):

```

1  -- Attempt 1: Trying to sell Batch 5003 which FAILED its lab
   test
2  INSERT INTO FG_Transaction (Invoice_No, Batch_No, Sale_Qty, Val
   )
3  VALUES (2020, 5003, 1000, 50000.00);
4
5  -- Attempt 2: Trying to sell an unverified Batch 5000 (No lab
   records)
6  INSERT INTO FG_Transaction (Invoice_No, Batch_No, Sale_Qty, Val
   )
7  VALUES (2020, 5000, 1000, 50000.00);
8
9  -- Attempt 3: Selling a safe, PASSED batch successfully
10 INSERT INTO FG_Transaction (Invoice_No, Batch_No, Sale_Qty, Val
   )
11 VALUES (2020, 5001, 1000, 50000.00);

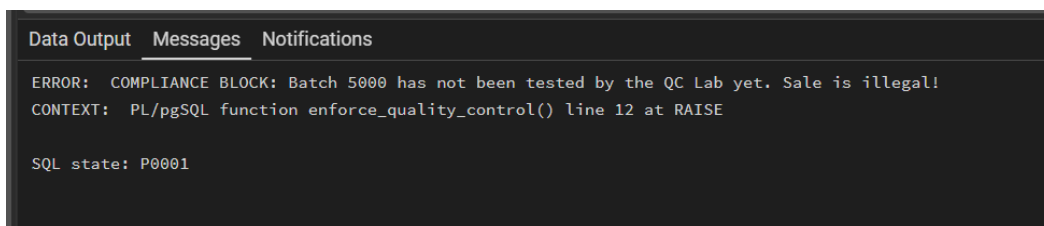
```

Test Outputs:



The screenshot shows a database console with three tabs: 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is selected, displaying an error message: 'ERROR: SAFETY BLOCK: Batch 5003 FAILED quality control. This batch must be quarantined, not sold!'. Below the error, the context is shown: 'CONTEXT: PL/pgSQL function enforce_quality_control() line 17 at RAISE'. At the bottom, the SQL state is listed as 'SQL state: P0001'.

Figure 10: Output 1: FDA Trigger blocking the sale of a FAILED batch (Batch 5003)



The screenshot shows a database console with three tabs: 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is selected, displaying an error message: 'ERROR: COMPLIANCE BLOCK: Batch 5000 has not been tested by the QC Lab yet. Sale is illegal!'. Below the error, the context is shown: 'CONTEXT: PL/pgSQL function enforce_quality_control() line 12 at RAISE'. At the bottom, the SQL state is listed as 'SQL state: P0001'.

Figure 11: Output 2: FDA Trigger blocking the sale of an UNVERIFIED batch (Batch 5000)

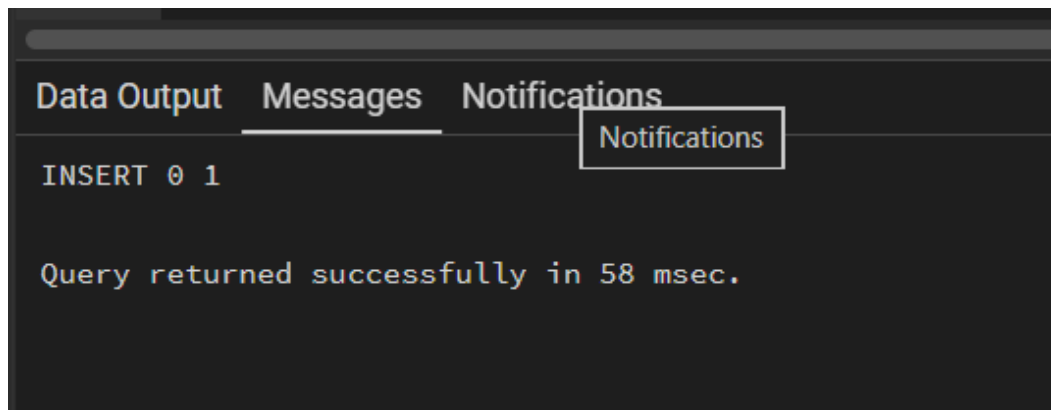


Figure 12: Output 3: Successful sale of a safe, PASSED batch (Batch 5001)

9.3 3. The Timeline Enforcer (trg_strict_batch_dates)

Execution Phase: BEFORE INSERT OR UPDATE on Batch

Why is this important?

- **Preventing Time Travel:** It acts as a strict chronological safeguard. If a data-entry worker accidentally types the wrong year, or attempts to fraudulently forward-date a batch to extend its shelf life, the database **blocks the entry immediately**.
- **Shelf-Life Compliance:** Distributors and hospitals will reject medicine that expires too quickly. This trigger mathematically guarantees that no medicine can even be registered in the system unless it possesses a **strict minimum of a 6-month viability window**, protecting the brand's supply chain standards.

```
1  --
2  -- FUNCTION: Strict Batch Date Enforcer
3  -- DESCRIPTION: Prevents setting manufacturing dates in the
4  --               future, and
5  --               ensures a minimum 6-month shelf life.
6  --
7  CREATE OR REPLACE FUNCTION enforce_batch_dates()
8  RETURNS TRIGGER
9  LANGUAGE plpgsql
10 AS $$
11 BEGIN
12     -- Rule 1: Prevent "Time Travel" (Manufacturing in the
13     --       future)
14     IF NEW.Mfg_Date > CURRENT_DATE THEN
15         RAISE EXCEPTION 'TIME_TRAVEL_DETECTED: You cannot set a
16             Manufacturing Date(%) in the future!', NEW.
17             Mfg_Date;
18     END IF;
19
20     -- Rule 2: Ensure minimum 6-month shelf life
21     -- (We add 6 months to the Mfg_Date and compare it to
22     --       Exp_Date)
23     IF NEW.Exp_Date < (NEW.Mfg_Date + INTERVAL '6 months') THEN
24         RAISE EXCEPTION 'QUALITY_BLOCK: The Expiry Date(%)
25             must be at least 6 months after the Manufacturing
26             Date(%).', NEW.Exp_Date, NEW.Mfg_Date;
27     END IF;
28
29     -- If both rules pass, allow the INSERT or UPDATE to finish
30     RETURN NEW;
31 END;
32 $$;
33
34 -- Bind the trigger to fire BEFORE BOTH Inserts and Updates!
35 CREATE TRIGGER trg_strict_batch_dates
```

```

30 BEFORE INSERT OR UPDATE ON Batch
31 FOR EACH ROW
32 EXECUTE FUNCTION enforce_batch_dates();

```

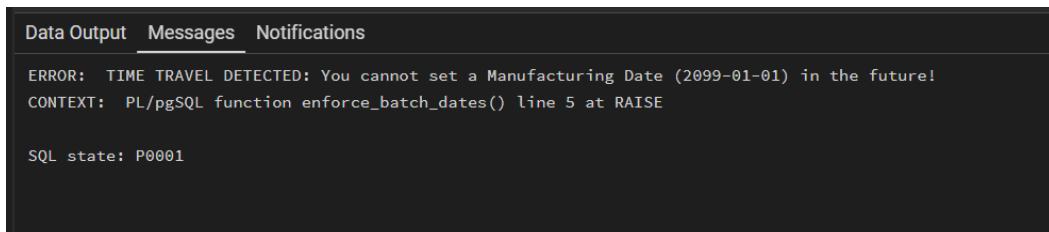
Live Test Cases (Triggering the Date Blocks):

```

1  -- Attempt 1: Triggering Rule 1 (Time Travel)
2  UPDATE Batch
3  SET Mfg_Date = '2099-01-01'
4  WHERE Batch_No = 5001;
5
6  -- Attempt 2: Triggering Rule 2 (Minimum 6-month shelf life
   failure)
7  INSERT INTO Batch (Batch_No, Batch_Size, Mfg_Date, Exp_Date,
   Product_ID, Stock_Qty, UT_Q_A)
8  VALUES (9999, 10000, '2024-01-01', '2024-02-01', 'PRD001',
   10000, 'UT');

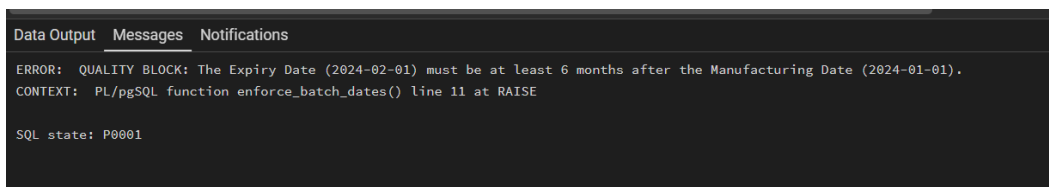
```

Test Outputs:



The screenshot shows a database interface with tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is selected, displaying an error message: 'ERROR: TIME TRAVEL DETECTED: You cannot set a Manufacturing Date (2099-01-01) in the future!'. Below this, the context is shown as 'CONTEXT: PL/pgSQL function enforce_batch_dates() line 5 at RAISE'. At the bottom, the SQL state is listed as 'SQL state: P0001'.

Figure 13: Output 1: Database rejecting a future Manufacturing Date



The screenshot shows a database interface with tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is selected, displaying an error message: 'ERROR: QUALITY BLOCK: The Expiry Date (2024-02-01) must be at least 6 months after the Manufacturing Date (2024-01-01)'. Below this, the context is shown as 'CONTEXT: PL/pgSQL function enforce_batch_dates() line 11 at RAISE'. At the bottom, the SQL state is listed as 'SQL state: P0001'.

Figure 14: Output 2: Database rejecting a batch with insufficient shelf life

9.4 4. The Silent Auditor (trg_audit_qc_changes)

Execution Phase: AFTER UPDATE OR DELETE on Product_Quality_Check

Why is this important?

- **FDA 21 CFR Part 11 Compliance:** Federal regulations mandate that critical electronic pharmaceutical records cannot be altered or destroyed without generating a permanent, trackable digital paper trail.
- **Immutable Surveillance:** If an employee attempts to secretly change a failing lab grade to a passing one, or deletes a record entirely to hide a mistake, this trigger acts as a hidden security camera. It silently captures the original data, the tampered data, the exact action taken, and the database user's credentials, depositing them into a hidden vault (QC_Audit_Log) for forensic review.

```
1  --
2  -- FUNCTION: The Silent Auditor
3  -- DESCRIPTION: Tracks any modifications or deletions of Lab
4  -- Results.
5  --
6  CREATE OR REPLACE FUNCTION track_qc_changes()
7  RETURNS TRIGGER
8  LANGUAGE plpgsql
9  AS $$
10 BEGIN
11     -- SCENARIO A: Someone is trying to UPDATE a lab result
12     IF TG_OP = 'UPDATE' THEN
13         -- If the result actually changed, log the evidence!
14         IF OLD.Results IS DISTINCT FROM NEW.Results THEN
15             INSERT INTO QC_Audit_Log (Report_ID, Action_Type,
16                                     Old_Result, New_Result, Changed_By)
17             VALUES (OLD.Report_ID, 'UPDATE', OLD.Results, NEW.
18                     Results, CURRENT_USER);
19         END IF;
20         RETURN NEW;
21
22     -- SCENARIO B: Someone is trying to totally DELETE a lab
23     -- record
24     ELSIF TG_OP = 'DELETE' THEN
25         INSERT INTO QC_Audit_Log (Report_ID, Action_Type,
26                                 Old_Result, New_Result, Changed_By)
27         VALUES (OLD.Report_ID, 'DELETE', OLD.Results, 'RECORD_
28                 DESTROYED', CURRENT_USER);
29         RETURN OLD;
30     END IF;
31 END;
32 $$;
33
34 -- Bind the trigger to listen AFTER someone touches the QC
35 -- table
36 CREATE TRIGGER trg_audit_qc_changes
37 AFTER UPDATE OR DELETE ON Product_Quality_Check
```

```

31 FOR EACH ROW
32 EXECUTE FUNCTION track_qc_changes();

```

Live Test Cases (Triggering the Audit Trail):

```

1  -- Attempt 1: An employee secretly changes a FAILED test to
   PASSED
2  UPDATE Product_Quality_Check
3  SET Results = 'PASSED'
4  WHERE Report_ID = 'PQC003';
5
6  -- Attempt 2: An employee tries to destroy a record entirely
7  DELETE FROM Product_Quality_Check
8  WHERE Report_ID = 'PQC005';
9
10 -- Forensic Review: Checking the hidden audit ledger for
   evidence
11 SELECT * FROM QC_Audit_Log;

```

Test Output:

Data Output Messages Notifications							
audit_id [PK] integer	report_id character varying (20)	action_type character varying (10)	old_result character varying (50)	new_result character varying (50)	changed_by character varying (50)	change_date timestamp without time zone	
1	1 PQC003	UPDATE	FAILED	PASSED	postgres	2026-05-16 16:10:16.005322	
2	2 PQC005	DELETE	PASSED	RECORD DESTROYED	postgres	2026-05-16 16:11:09.077744	

Figure 15: Output: The shadow ledger capturing the unauthorized data modifications and deletion

10 Data Abstraction: Enterprise Views

In a highly normalized database, business intelligence (BI) tools and frontend dashboards often require data that spans multiple complex joins. To optimize performance and restrict direct access to core transactional tables, PharmaChain-DB utilizes native SQL Views. These act as secure, pre-compiled virtual tables that serve real-time analytics to specific factory departments.

10.1 1. Procurement Dashboard: Low Stock Alerts (v_inventory_shortage)

Why is this important?

- **Automated Supply Chain Triggers:** Supply chain managers do not have time to manually calculate the remaining stock of hundreds of chemicals. This view acts as a daily "To-Do List" for the purchasing team, instantly filtering out healthy stock and only showing materials in the danger zone.
- **Preventing Factory Halts:** By calculating the exact difference between current stock and the defined `Reorder_Level`, it tells the procurement team exactly how many units they need to buy to prevent a total manufacturing shutdown.
- **Data Security:** It allows the purchasing department's software to see inventory deficits without ever granting them direct `UPDATE` or `DELETE` permissions on the sensitive `Warehouse` table.

```
1  --
2  -- VIEW: Procurement Dashboard (Low Stock Alert)
3  -- DESCRIPTION: Instantly shows the purchasing team exactly
4  --               which
5  --               materials have dropped below their safe reorder
6  --               limit.
7  --
8  CREATE OR REPLACE VIEW v_inventory_shortage AS
9  SELECT
10     w.Item_ID,
11     m.Material_Name,
12     m.Material_Type,
13     w.Stock AS Current_Stock,
14     m.Reorder_Level AS Minimum_Required,
15     (m.Reorder_Level - w.Stock) AS Units_To_Order
16 FROM Warehouse w
17 JOIN Material_Master m ON w.Material_ID = m.Material_ID
18 WHERE w.Stock <= m.Reorder_Level;
```

Live Test Case (Viewing the Dashboard):

```
1  -- The purchasing software executes this simple query every
2  -- morning
3  SELECT * FROM v_inventory_shortage;
```

Data Output Messages Notifications						
Showing rows: 1 to 19						
	item_id bigint	material_name character varying (30)	material_type character varying (20)	current_stock numeric (10)	minimum_required numeric (10)	units_to_order numeric
	2	Omeprazole	Drug	2000	2000	0
	3	Azithromycin Dihydrate	Drug	1500	1500	0
	4	Atorvastatin	Drug	1000	1000	0
	5	Amoxicillin Trihydrate	Drug	3000	3000	0
	6	Diphenhydramine	Drug	1000	1000	0
	9	Sucrose	Excipient	15000	15000	0
	10	Starch	Excipient	20000	20000	0
	11	Isopropyl Alcohol	Excipient	5000	5000	0
	12	Vitamin A	Supplement	500	500	0
0	13	Vitamin D3	Supplement	500	500	0
1	14	Lactose Monohydrate	Excipient	5000	5000	0
2	16	Talc	Excipient	3000	3000	0
3	17	Ciprofloxacin	Drug	1500	1500	0
4	18	Ibuprofen	Drug	4000	4000	0
5	19	Glimepiride	Drug	1000	1000	0
6	20	Propylene Glycol	Excipient	2500	2500	0
7	7	Sodium Starch Glycol...	Excipient	7500	8000	500
8	8	Methyl Cellulose	Excipient	9800	10000	200
9	15	Magnesium Stearate	Excipient	1950	2000	50

Figure 16: Output: The Procurement Dashboard highlighting urgent chemical shortages

Test Output:

10.2 2. FDA End-to-End Batch Traceability (v_fda_batch_traceability)

Why is this important?

- **Instant Audit Readiness:** During an FDA or regulatory audit, inspectors demand the complete lifecycle of a medicine. This view instantly aggregates data across six different tables to show exactly when a pill was made, what raw chemicals were mixed into it, its final lab grade, and how much was sold.
- **Surgical Product Recalls:** If a specific raw material supplier issues a contamination warning, this view allows executives to instantly identify every single finished medicine batch that contains that specific chemical, preventing a massive, uncoordinated market recall.
- **Cross-Departmental Visibility:** It bridges the gap between Manufacturing, Quality Control, and Sales. Instead of writing complex, 7-table JOIN queries every time management needs a report, they can query this view like a simple spreadsheet.

```
1  --
2
3  -- VIEW: FDA End-to-End Batch Traceability
4  -- DESCRIPTION: A complete lifecycle map of every batch,
5  --               showing its
6  --               ingredients, QC status, and market sales.
7  --
8  CREATE OR REPLACE VIEW v_fda_batch_traceability AS
9  SELECT
10     B.Batch_No ,
11     PM.Product_Name ,
12     B.Mfg_Date ,
13     B.Exp_Date ,
14     COALESCE(QC.Results, 'UNTESTED') AS QC_Status ,
15     STRING_AGG(DISTINCT MM.Material_Name, ',') AS
16     Raw_Materials_Used ,
17     COALESCE(SUM(FGT.Sale_Qty), 0) AS Total_Sold_To_Market
18 FROM Batch B
19 JOIN Product_Master PM ON B.Product_ID = PM.Product_ID
20 LEFT JOIN Product_Quality_Check QC ON B.Batch_No = QC.Batch_No
21 LEFT JOIN Material_Dispensing MD ON B.Batch_No = MD.Batch_No
22 LEFT JOIN Warehouse W ON MD.Item_ID = W.Item_ID
23 LEFT JOIN Material_Master MM ON W.Material_ID = MM.Material_ID
24 LEFT JOIN FG_Transaction FGT ON B.Batch_No = FGT.Batch_No
25 GROUP BY
26     B.Batch_No ,
27     PM.Product_Name ,
28     B.Mfg_Date ,
29     B.Exp_Date ,
30     QC.Results;
```

Live Test Case (Viewing the Traceability Ledger):


```

1  -- Quality Assurance executes this to generate a complete
    compliance report
2  SELECT * FROM v_fda_batch_traceability;

```

Test Output:

Data Output Messages Notifications							
Showing rows: 1 to 20 Page No: 1 of 1							
	batch_no numeric (10)	product_name character varying (20)	mfg_date date	exp_date date	qc_status character varying	raw_materials_used text	total_sold_to_market numeric
1	5001	Parabufen	2023-03-01	2026-05-31	PASSED	Magnesium Stearate, Methyl Cellulose, Paracetamol API, Sodium Starch Glyco...	84000
2	5002	Omaergy	2023-03-02	2026-07-15	PASSED	Omeprazole	10000
3	5003	Exicof	2023-03-03	2024-03-02	FAILED	Diphenhydramine	0
4	5004	Atorvastatin 20	2023-03-04	2027-05-16	RECALLED	Atorvastatin	15000
5	5005	Amoxyn	2023-03-05	2024-03-04	UNTESTED	Amoxycillin Trihydrate	20000
6	5006	Vitarich	2023-03-06	2026-03-05	PASSED	Vitamin A	25000
7	5007	Zithrol 500	2023-03-07	2024-03-06	PASSED	Azithromycin Dihydrate	10000
8	5008	Aller-D	2023-03-08	2024-03-07	PASSED	Diphenhydramine	10000
9	5009	Cipro-Max	2023-03-09	2025-03-08	PASSED	Ciprofloxacin	30000
10	5010	Ibupro-G	2023-03-10	2026-03-09	PASSED	Ibuprofen	20000
11	5011	Glimeta 2	2023-03-11	2025-03-10	FAILED	Glimepiride	12000
12	5012	Para-500	2023-03-12	2025-03-11	PASSED	Paracetamol API	40000
13	5013	Para-650	2023-03-13	2025-03-12	PASSED	Paracetamol API	35000
14	5014	Vit-D Core	2023-03-14	2025-03-13	PASSED	Vitamin D3	5000
15	5015	Ome-Protect	2023-03-15	2025-03-14	PASSED	Omeprazole	10000
16	5016	Amox-500	2023-03-16	2025-03-15	PASSED	Amoxycillin Trihydrate	15000
17	5017	Atorva-10	2023-03-17	2025-03-16	PASSED	Atorvastatin	15000
18	5018	Zithro-Susp	2023-03-18	2024-03-17	PASSED	Azithromycin Dihydrate	7000
19	5019	Glime-M	2023-03-19	2025-03-18	FAILED	Glimepiride	12000
20	5020	Cipro-Drops	2023-03-20	2024-03-19	PASSED	Ciprofloxacin	4000

Figure 17: Output: The complete lifecycle and market trace of manufactured batches

10.3 3. Expiry Risk & Inventory Liability Dashboard (v_inventory_expiry_risk)

Why is this important?

- **Financial Liability Mitigation:** Pharmaceutical products have strict shelf lives. This view acts as an early warning system, allowing the sales team to heavily discount or push batches that are approaching expiration before they become a total financial loss.
- **Automated Risk Categorization:** Instead of relying on employees to manually calculate dates, the database natively categorizes inventory into actionable status codes (SAFE, WARNING, CRITICAL, EXPIRED) based on real-time chronological math.
- **Accurate "Floor Stock" Tracking:** By programmatically subtracting all completed sales from the initial manufactured batch size, executives know exactly how many units are physically sitting unsold in the warehouse at any given second.

```
1  --
2  -- VIEW: Expiry Risk & Inventory Liability Dashboard
3  -- DESCRIPTION: Tracks unsold finished goods, calculates days
4  --                expiry, and flags high-risk inventory using
5  --                status codes.
6  --
7  CREATE OR REPLACE VIEW v_inventory_expiry_risk AS
8  SELECT
9      B.Batch_No ,
10     PM.Product_Name ,
11     B.Exp_Date ,
12     (B.Exp_Date - CURRENT_DATE) AS Days_Remaining ,
13
14     -- Professional Status Codes
15     CASE
16         WHEN (B.Exp_Date - CURRENT_DATE) < 0 THEN 'EXPIRED-DO
17             NOT SELL'
18         WHEN (B.Exp_Date - CURRENT_DATE) <= 30 THEN 'CRITICAL-
19             UNDER 30 DAYS'
20         WHEN (B.Exp_Date - CURRENT_DATE) <= 90 THEN 'WARNING-
21             UNDER 90 DAYS'
22         ELSE 'SAFE'
23     END AS Risk_Status ,
24
25     B.Stock_Qty AS Manufactured_Qty ,
26     COALESCE(SUM(FGT.Sale_Qty), 0) AS Total_Sold_Qty ,
27
28     -- Calculate exactly how many units are sitting unsold on
29     -- the shelf
30     (B.Stock_Qty - COALESCE(SUM(FGT.Sale_Qty), 0)) AS
31     Unsold_Inventory
```

```

27
28 FROM Batch B
29 JOIN Product_Master PM ON B.Product_ID = PM.Product_ID
30 LEFT JOIN FG_Transaction FGT ON B.Batch_No = FGT.Batch_No
31
32 GROUP BY
33     B.Batch_No ,
34     PM.Product_Name ,
35     B.Exp_Date ,
36     B.Stock_Qty
37
38 -- ONLY show batches where we still have unsold inventory
39 HAVING (B.Stock_Qty - COALESCE(SUM(FGT.Sale_Qty), 0)) > 0
40
41 -- Sort the most dangerous (expiring soonest) to the top
42 ORDER BY Days_Remaining ASC;

```

Live Test Cases (Setting up Expiry Risks & Viewing the Dashboard):

```

1 -- 1. Test "CRITICAL" (Set Expiry to 15 days from today)
2 UPDATE Batch
3 SET Exp_Date = CURRENT_DATE + INTERVAL '15_days'
4 WHERE Batch_No = 5001;
5
6 -- 2. Test "WARNING" (Set Expiry to 60 days from today)
7 UPDATE Batch
8 SET Exp_Date = CURRENT_DATE + INTERVAL '60_days'
9 WHERE Batch_No = 5002;
10
11 -- 3. Test "SAFE" (Set Expiry to 1 year from today)
12 UPDATE Batch
13 SET Exp_Date = CURRENT_DATE + INTERVAL '1_year'
14 WHERE Batch_No = 5004;
15
16 -- Generate the Expiry Risk Report
17 SELECT * FROM v_inventory_expiry_risk;

```

Test Output:

Data Output Messages Notifications								
Showing rows: 1 to 19 Page N								
	batch_no numeric (10)	product_name character varying (20)	exp_date date	days_remaining integer	risk_status text	manufactured_qty numeric (10)	total_sold_qty numeric	unsold_inventory numeric
1	5003	Exicof	2024-03-02	-806	EXPIRED - DO NOT SELL	9500	0	9500
2	5005	Amoxyn	2024-03-04	-804	EXPIRED - DO NOT SELL	39000	20000	19000
3	5007	Zithrol 500	2024-03-06	-802	EXPIRED - DO NOT SELL	14900	10000	4900
4	5008	Aller-D	2024-03-07	-801	EXPIRED - DO NOT SELL	19800	10000	9800
5	5018	Zithro-Susp	2024-03-17	-791	EXPIRED - DO NOT SELL	14500	7000	7500
6	5020	Cipro-Drops	2024-03-19	-789	EXPIRED - DO NOT SELL	9500	4000	5500
7	5009	Cipro-Max	2025-03-08	-435	EXPIRED - DO NOT SELL	59000	30000	29000
8	5011	Glimeta 2	2025-03-10	-433	EXPIRED - DO NOT SELL	24500	12000	12500
9	5012	Para-500	2025-03-11	-432	EXPIRED - DO NOT SELL	79000	40000	39000
10	5013	Para-650	2025-03-12	-431	EXPIRED - DO NOT SELL	74000	35000	39000
11	5014	Vit-D Core	2025-03-13	-430	EXPIRED - DO NOT SELL	9800	5000	4800
12	5015	Ome-Protect	2025-03-14	-429	EXPIRED - DO NOT SELL	19500	10000	9500
13	5016	Amox-500	2025-03-15	-428	EXPIRED - DO NOT SELL	34000	15000	19000
14	5017	Atorva-10	2025-03-16	-427	EXPIRED - DO NOT SELL	29000	15000	14000
15	5019	Glime-M	2025-03-18	-425	EXPIRED - DO NOT SELL	24000	12000	12000
16	5006	Vitarich	2026-03-05	-73	EXPIRED - DO NOT SELL	48000	25000	23000
17	5010	Ibupro-G	2026-03-09	-69	EXPIRED - DO NOT SELL	44000	20000	24000
18	5001	Parabufen	2026-05-31	14	CRITICAL - UNDER 30 DA...	49000	21000	28000
19	5002	Omsergy	2026-07-15	59	WARNING - UNDER 90 D...	24800	10000	14800

Figure 18: Output: The Expiry Risk Dashboard flagging batches nearing expiration

11 Complex Transactions: Stored Procedures

Architectural Concept: Stored Procedures vs. Triggers

While both Stored Procedures and Triggers execute business logic on the database server, their architectural roles are fundamentally different:

- **Triggers are Reactive (Event-Driven):** A trigger acts as a hidden background guard. It fires automatically in response to a DML event (INSERT, UPDATE, DELETE). The user or application does not explicitly call a trigger.
- **Stored Procedures are Proactive (On-Demand):** A stored procedure is a pre-compiled script called manually by an application or user via the CALL command. They are designed to group multiple, complex database operations into a single **atomic transaction**. If any part of the procedure fails, the entire transaction rolls back, preventing partial data updates.

11.1 1. The Emergency Quarantine Protocol (execute_product_recall)

Why is this important?

- **Rapid Crisis Response:** During a catastrophic medicine failure or FDA mandate, executives do not have time to manually update multiple tables. This procedure allows a manager to execute a total factory quarantine with a single line of code.
- **Atomic Multi-Table Transactions:** This script executes a highly orchestrated sequence: it checks if the batch exists, verifies physical stock, logs the legal recall event, zeroes out the physical warehouse inventory, and overwrites the Quality Control lab records. Grouping this into a procedure ensures that all steps happen together, or none happen at all.
- **Frontend Abstraction:** The application developers do not need to know the complex relational schema of the database. They simply pass the Recall_ID, Batch_No, and Reason to the procedure, and the database handles the heavy lifting safely.

```
1  --
2  -- PROCEDURE: Execute Emergency Product Recall (Tailored)
3  -- DESCRIPTION: Integrates exactly with the user's
4  --               Product_Recall table,
5  --               enforcing all custom constraints and VARCHAR
6  --               keys.
7  --
8  CREATE OR REPLACE PROCEDURE execute_product_recall(
9      p_recall_id VARCHAR(20),
10     p_batch_no  NUMERIC,
11     p_reason    VARCHAR(255)
12 )
13 LANGUAGE plpgsql
```

```

13 AS $$
14 DECLARE
15     v_unsold_qty NUMERIC;
16 BEGIN
17     -- 1. Find out exactly how many units are currently in the
18     warehouse
19     SELECT Stock_Qty INTO v_unsold_qty
20     FROM Batch
21     WHERE Batch_No = p_batch_no;
22
23     -- Safety Check A: Does this batch exist?
24     IF v_unsold_qty IS NULL THEN
25         RAISE EXCEPTION 'RECALL_FAILED: Batch % does not exist
26         in the system.', p_batch_no;
27     END IF;
28
29     -- Safety Check B: Satisfy your CHECK (Qty_Recalled > 0)
30     constraint
31     IF v_unsold_qty <= 0 THEN
32         RAISE EXCEPTION 'RECALL_FAILED: Batch % has 0 stock in
33         the warehouse. Nothing left to quarantine!',
34         p_batch_no;
35     END IF;
36
37     -- 2. Log the event into YOUR exact table
38     INSERT INTO Product_Recall (Recall_ID, Batch_No,
39     Date_Initiated, Reason, Qty_Recalled)
40     VALUES (p_recall_id, p_batch_no, CURRENT_DATE, p_reason,
41     v_unsold_qty);
42
43     -- 3. Zero out the warehouse stock
44     UPDATE Batch
45     SET Stock_Qty = 0
46     WHERE Batch_No = p_batch_no;
47
48     -- 4. Overwrite the Quality Control lab result to
49     quarantine the batch
50     UPDATE Product_Quality_Check
51     SET Results = 'RECALLED'
52     WHERE Batch_No = p_batch_no;
53
54 END;
55 $$;

```

Live Test Case (Executing a Corporate Recall):

```

1 -- Execute the emergency procedure for Batch 5004
2 CALL execute_product_recall(
3     'REC-5004-A',
4     5004,
5     'CRITICAL: Stability test failed at 6 months. Active
6     ingredient degraded.'

```

```

6 );
7
8 -- View the official registry to confirm the transaction
   succeeded
9 SELECT * FROM Product_Recall;

```

Test Output:

	recall_id [PK] character varying (20)	batch_no numeric (10)	date_initiated date	reason character varying (255)	qty_recalled numeric (10)
1	REC001	5003	2023-05-01	Failed viscosity stability tests.	5000
2	REC002	5011	2023-05-02	Assay degraded below 90% in market.	12000
3	REC003	5019	2023-05-03	Contamination warning from raw supplier.	12000
4	REC004	5001	2023-05-04	Packaging label misprint.	1000
5	REC005	5002	2023-05-05	Seal integrity failure reported.	500
6	REC006	5004	2023-05-06	Customer reports of bad odor.	2000
7	REC007	5005	2023-05-07	pH level dropped out of spec.	3000
8	REC008	5006	2023-05-08	Voluntary recall - updated formulation.	4000
9	REC009	5007	2023-05-09	Failed 3-month dissolution test.	1500
10	REC010	5008	2023-05-10	Capsule shells brittle.	2500
11	REC011	5009	2023-05-11	Incorrect expiry date printed.	5000
12	REC012	5010	2023-05-12	Discoloration of tablets.	1000
13	REC013	5012	2023-05-13	Failed friability test post-shipping.	8000
14	REC014	5013	2023-05-14	Blister pack foil delamination.	4500
15	REC015	5014	2023-05-15	Sub-potent assay reported by distributor.	500
16	REC016	5015	2023-05-16	Cross-contamination suspected.	2000
17	REC017	5016	2023-05-17	Failed microbial limits test.	3500
18	REC018	5017	2023-05-18	Foreign particle found in bottle.	100
19	REC019	5018	2023-05-19	Syrup crystallized prematurely.	3000
20	REC020	5020	2023-05-20	Dropper malfunction reported.	1500
21	REC-5004-A	5004	2026-05-16	CRITICAL: Stability test failed at 6 months. Active ingredient degra...	29500

Figure 19: Output: Successful execution of the Emergency Product Recall stored procedure

12 BCNF Verification Table

The following table demonstrates the mathematical verification that every relation in the PharmaChain-DB architecture satisfies Boyce-Codd Normal Form (BCNF). For each entity, the minimal functional dependency (FD) set is identified. Because every determinant in the architecture is a Candidate Key (or a strict superset thereof), no partial or transitive dependencies exist, ensuring absolute data integrity.

Table 1: BCNF Verification Table: Pharmaceutical Industry Management System

Relation	Minimal FD Set	Candidate Key	BCNF Proof
Material_Master	Material_ID $\rightarrow$ {Material_Name, Material_Type, Storage_Condition, Shelf_Life, Therapeutic_Category, Material_State, isHazardous, isInflammable, UOM, Reorder_Level}	Material_ID	The only FD is of the form $X \rightarrow Y$. Since Material_ID is the Candidate Key, it is a superkey.
Account_Master	Account_No $\rightarrow$ {Account_Name, Phone_No, Address, Account_Type}	Account_No	All FDs are of the form Account_No $\rightarrow Y$. Account_No is the Candidate Key and thus a superkey.
Employee_Master	Emp_ID $\rightarrow$ {Emp_Name, Department, Role, Hire_Date}	Emp_ID	The only FD is Emp_ID $\rightarrow Y$. The LHS is the Candidate Key and thus a superkey.
Equipment_Master	Equipment_ID $\rightarrow$ {Equipment_Name, Equipment_Type, Last_Calibration_Date, Status}	Equipment_ID	The only FD is Equipment_ID $\rightarrow Y$. The LHS is the Candidate Key and thus a superkey.
Product_Master	Product_ID $\rightarrow$ {Product_Name, Generic_Name, Product_Type, Packing_Type, Packing_Size, SalableorSample, GenericorBranded}	Product_ID	The only FD is Product_ID $\rightarrow Y$. The LHS is the Candidate Key and thus a superkey.

Continued on next page

Table 1 – *Continued from previous page*

Relation	Minimal FD Set	Candidate Key	BCNF Proof
Transactions	Invoice_No $\rightarrow$ {Transaction_Date, Currency, Transaction_Type, Paid_Received, Account_No, Total_Value}	Invoice_No	The only FD is Invoice_No $\rightarrow$ Y. The LHS is the Candidate Key and thus a superkey.
Supplier_Contract	Contract_ID $\rightarrow$ {Account_No, Material_ID, Agreed_Price, Valid_Until}	Contract_ID	All FDs are of the form Contract_ID $\rightarrow$ Y. Contract_ID is the Candidate Key and thus a superkey.
Warehouse	Item_ID $\rightarrow$ {Material_ID, Invoice_No, UT_Q_A, Stock}	Item_ID	The only FD is Item_ID $\rightarrow$ Y. Item_ID is the Candidate Key and thus a superkey.
RM_Transaction	{Invoice_No, Item_ID} $\rightarrow$ {RM_Qty, Val}	{Invoice_No, Item_ID}	All FDs are of the form {Invoice_No, Item_ID} $\rightarrow$ Y. The LHS is the Candidate Key and thus a superkey.
Material_Quality_Check	Report_ID $\rightarrow$ {Item_ID, Analysis_Date, Analyst_Name, Sample_Size, Test, Limits, Results, Emp_ID}	Report_ID	The only FD is Report_ID $\rightarrow$ Y. Report_ID is the Candidate Key and thus a superkey.
Formula_Master	{Product_ID, Material_ID} $\rightarrow$ {Weight_per_tablet}	{Product_ID, Material_ID}	The only FD is {Product_ID, Material_ID} $\rightarrow$ Y. The LHS is the Candidate Key and thus a superkey.

Continued on next page

Table 1 – *Continued from previous page*

Relation	Minimal FD Set	Candidate Key	BCNF Proof
Batch	Batch_No $\rightarrow$ {Batch_Size, Mfg_Date, Exp_Date, Product_ID, Stock_Qty, UT_Q_A, Yield_Percentage}	Batch_No	The only FDs are of the form Batch_No $\rightarrow$ Y. Batch_No is the Candidate Key and thus a superkey.
Material_Dispensing	{Batch_No, Item_ID} $\rightarrow$ {Quantity_Issued}	{Batch_No, Item_ID}	The only FD is {Batch_No, Item_ID} $\rightarrow$ Y. The LHS is the Candidate Key and thus a superkey.
Production_Log	Log_ID $\rightarrow$ {Batch_No, Equipment_ID, Emp_ID, Process_Stage, Start_Time, End_Time}	Log_ID	The only FD is Log_ID $\rightarrow$ Y. Log_ID is the Candidate Key and thus a superkey.
Product_Quality_Check	Report_ID $\rightarrow$ {Batch_No, Analysis_Date, Analyst_Name, Sample_Size, Process_State, Test, Limits, Results, Emp_ID}	Report_ID	The only FD is Report_ID $\rightarrow$ Y. Report_ID is the Candidate Key and thus a superkey.
FG_Transaction	{Invoice_No, Batch_No} $\rightarrow$ {Sale_Qty, Val}	{Invoice_No, Batch_No}	All FDs are of the form {Invoice_No, Batch_No} $\rightarrow$ Y. The LHS is the Candidate Key and thus a superkey.
Product_Recall	Recall_ID $\rightarrow$ {Batch_No, Date_Initiated, Reason, Qty_Recalled}	Recall_ID	The only FD is Recall_ID $\rightarrow$ Y. The LHS is the Candidate Key and thus a superkey.

Continued on next page

Table 1 – *Continued from previous page*

Relation	Minimal FD Set	Candidate Key	BCNF Proof
Maintenance_Log	Maintenance_ID $\rightarrow$ {Equipment_ID, Emp_ID, Maintenance_Date, Cost}	Maintenance_ID	The only FD is Maintenance_ID $\rightarrow$ Y. The LHS is the Candidate Key and thus a superkey.
QC_Audit_Log	Audit_ID $\rightarrow$ {Report_ID, Action_Type, Old_Result, New_Result, Changed_By, Change_Date}	Audit_ID	The only FD is Audit_ID $\rightarrow$ Y. The LHS is the Candidate Key and thus a superkey.

13 Future Scope and Next Steps

While the current PharmaChain-DB architecture successfully achieves strict normalization and data integrity, a database must evolve as the company grows. The following practical database improvements outline our next steps for making the system faster, safer, and easier to connect to other software.

1. Automated Email and SMS Alerts

- **Active Notifications:** Right now, employees have to manually check the database views to see if raw materials are running low.
- **Database Triggers to Email:** In the future, we can configure the database (using tools like `pg_notify`) to automatically send an email or text message to the purchasing manager the exact second a chemical drops below its safe reorder level.

2. Data Archiving for Faster Performance

- **Managing Database Growth:** After five or ten years, tables like `Production_Log` and `Transactions` will contain millions of rows, which can slow down daily queries.
- **Historical Tables:** We plan to build automated archiving scripts that move very old, completed medicine batches into separate "History" tables. This keeps the main active tables small and lightning-fast, while still legally saving the old data.

3. Connecting to Web and Mobile Apps (API Integration)

- **Beyond SQL Queries:** Currently, users interact with the data by running SQL scripts directly.
- **Building a REST API:** The next step is to wrap the database in a secure API. This will allow frontend web developers to easily build a simple website or mobile app where factory workers can just tap buttons on a screen to insert data, rather than knowing any code.

4. Materialized Views for Heavy Reports

- **Speeding Up Dashboards:** Our `v_fda_batch_traceability` view joins seven different tables together. If executives run this report multiple times a day on a massive database, it could heavily tax the server's CPU.
- **Pre-saving the Math:** We can upgrade standard views into "Materialized Views." This means the database will pre-calculate the heavy math overnight and save the results, so when the boss asks for the report in the morning, it loads instantly.

5. Automated Backups and Disaster Recovery

- **Protecting the Data:** If the factory's main server loses power or a hard drive crashes, years of pharmaceutical records could be lost.
- **Nightly Snapshots:** We will implement automated database cron jobs (scheduled tasks) that safely copy the entire database every night and send the backup file to a secure, off-site location so it can always be restored.